claude-windows / dbc4cdae-04f6-4d06-af90-397bbce3fc57

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\subagents\agent-a1a9b1f7403778625.jsonl`
- SHA-256: `e6565f2569cc74ab92362f14390fe48a0adbccd98dc1d151ee78e5573e52b007`
- Source modified: `2026-07-06T20:26:32+00:00`
- Imported at: `2026-07-08T15:59:36+00:00`
- project: `subagents`
- session_id: `dbc4cdae-04f6-4d06-af90-397bbce3fc57`

Transcript

1. user (2026-07-06T20:20:02.244Z)

Write a self-contained Python experiment script `exp_t2.py` for a torch-2.x GPU-resident-decode spike of the WASB shuttle detector. It will run on a Linux L4 GPU VM (NOT here). Output ONLY the final script in a ```python code block, plus a short notes section. Do not run anything.

Read these local sources first (the cloned WASB-SBDT repo)

`C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\scratchpad\WASB-SBDT\` ? especially:
- `src/models/hrnet.py` (the HRNet class + any `build_model`/factory), `src/models/__init__.py`
- `src/detectors/detector.py` (how the detector builds HRNet + does `torch.load`/`load_state_dict` + `run_tensor` + heatmap decode), `src/detectors/postprocessor.py`, `src/detectors/__init__.py`
- `src/configs/model/wasb.yaml` (HRNet architecture params) and `src/configs/dataset/badminton.yaml`
- `src/dataloaders/` (the exact preprocessing: ImageNet normalize constants, any /255 scale, resize interpolation, RGB order)

Figure out the MINIMAL way to construct the WASB HRNet and load weights **using WASB's own code** (import the model class / build fn; do NOT reimplement HRNet). If Hydra config composition is needed, prefer directly instantiating the HRNet class with the params from `wasb.yaml` to avoid Hydra runtime friction ? whichever is more robust.

Established facts (from a repo mapping ? trust these)

- Weights file `~/t2/wasb.pth.tar`: `torch.load(path, map_location=...)` returns `{"model_state_dict": <428-tensor state_dict>}`. Input = **9 channels (3 frames × RGB)**, model input **512×288 (WxH)**, output = **3 heatmaps (512×288)**. Preprocessing: resize each frame to 512×288, **RGB**, **ImageNet normalize** (confirm exact mean/std + whether /255 precedes it from the dataloader code).
- torch ?2.6 defaults `torch.load(weights_only=True)`; a plain tensor state_dict loads fine, but pass `weights_only=False` to be safe and note it.
- Sliding window = 3 consecutive frames, stride 1, concatenated on channel dim ? the 9-ch input.

VM file layout the script must assume

- `~/t2/WASB-SBDT` (repo), `~/t2/wasb.pth.tar` (weights), `~/t2/g1.mp4` (golden clip, 1080p ~59.94fps), `~/t2/g1_traj_baseline.csv` (cached baseline trajectory: columns `Frame,Visibility,X,Y`).
- Installed: torch 2.x (cuda), torchvision (transforms.v2), `torchcodec` (VideoDecoder, may support device="cuda"), `PyNvVideoCodec`, opencv-python-headless, numpy. `libnvcuvid` is present.

What exp_t2.py must do (print each clearly, unbuffered via `print(...,flush=True)`; process a SUBSET N=1500 f

1. **Q1 ? port:** build WASB HRNet under torch 2.x, `load_state_dict` the weights, `model.eval().cuda()`. Run one forward on a dummy `(1,9,288,512)` tensor. Print torch version, "WASB HRNet built + weights loaded OK", output shape. Wrap in try/except and print the full error if it fails (this is the key Q1 result).

2. **Q2 ? throughput** over N frames, three variants, print fps for each (frames/sec of the full decode?resize?normalize?HRNet-forward micro-pipeline, batch the 3-frame windows reasonably, e.g. batch=8 windows):
   a. **CPU front-end:** cv2.VideoCapture decode ? cv2 resize to 512×288 ? RGB ? ImageNet-normalize on CPU ? to CUDA tensor ? HRNet.
   b. **GPU-resident (torchcodec):** `torchcodec` VideoDecoder(device="cuda") ? frames already CUDA uint8 ? GPU resize (torchvision.transforms.v2.functional or F.interpolate) ? GPU normalize ? HRNet. If torchcodec cuda fails, catch and print why, then try device="cpu" torchcodec + GPU resize.
   c. **GPU-resident (PyNvVideoCodec):** decode to CUDA tensor via DLPack ? GPU resize + normalize ? HRNet. If it fails, catch + print why.
3. **Q3 ? quality parity:** for the first ~200 frames, compare the **HRNet heatmap output** and the **argmax shuttle (x,y)** between variant (a) CPU and whichever GPU variant worked: report mean|?| and max|?| of the heatmaps, and the median/max pixel distance between argmax positions (mapped back to 512×288). This shows whether GPU decode+resize changes the detection.
4. Print a final SUMMARY block: Q1 pass/fail, fps table (a/b/c), Q3 heatmap mean|?| + argmax median px distance.

Robustness: each variant in its own try/except so one failure doesn't abort the others; if a GPU decoder is unavailable, print the reason and continue. Assume the 3-frame window's heatmap for the "current" frame is channel index 1 (middle) ? but read detector.py to confirm which output channel is used for the position; note your assumption.

Return the complete `exp_t2.py` plus a short "notes / assumptions / what to check" section (esp. the normalization constants and which heatmap channel ? position).

2. assistant (2026-07-06T20:20:09.952Z)

I'll read the WASB-SBDT sources first to understand the exact model construction and preprocessing, then write the script.

3. user (2026-07-06T20:20:14.043Z)

```
total 49
drwxr-xr-x 1 avidu 197609     0 Jul  7 01:49 .
drwxr-xr-x 1 avidu 197609     0 Jul  7 01:49 ..
drwxr-xr-x 1 avidu 197609     0 Jul  7 01:49 .git
-rw-r--r-- 1 avidu 197609    37 Jul  7 01:49 .gitignore
-rw-r--r-- 1 avidu 197609  1407 Jul  7 01:49 Dockerfile
-rw-r--r-- 1 avidu 197609 11935 Jul  7 01:49 GET_STARTED.md
-rw-r--r-- 1 avidu 197609  1108 Jul  7 01:49 LICENSE.md
-rw-r--r-- 1 avidu 197609  4481 Jul  7 01:49 MODEL_ZOO.md
-rw-r--r-- 1 avidu 197609  2169 Jul  7 01:49 README.md
drwxr-xr-x 1 avidu 197609     0 Jul  7 01:49 src
---SRC---
total 45
drwxr-xr-x 1 avidu 197609   0 Jul  7 01:49 .
drwxr-xr-x 1 avidu 197609   0 Jul  7 01:49 ..
drwxr-xr-x 1 avidu 197609   0 Jul  7 01:49 configs
drwxr-xr-x 1 avidu 197609   0 Jul  7 01:49 dataloaders
drwxr-xr-x 1 avidu 197609   0 Jul  7 01:49 datasets
drwxr-xr-x 1 avidu 197609   0 Jul  7 01:49 detectors
drwxr-xr-x 1 avidu 197609   0 Jul  7 01:49 losses
-rw-r--r-- 1 avidu 197609 638 Jul  7 01:49 main.py
drwxr-xr-x 1 avidu 197609   0 Jul  7 01:49 models
drwxr-xr-x 1 avidu 197609   0 Jul  7 01:49 optimizers
drwxr-xr-x 1 avidu 197609   0 Jul  7 01:49 runners
drwxr-xr-x 1 avidu 197609   0 Jul  7 01:49 setup_scripts
drwxr-xr-x 1 avidu 197609   0 Jul  7 01:49 trackers
drwxr-xr-x 1 avidu 197609   0 Jul  7 01:49 utils
```

4. assistant (2026-07-06T20:20:19.364Z)

Now let me read the key model and detector files in parallel.

5. user (2026-07-06T20:20:22.402Z)

```
# ------------------------------------------------------------------------------
# Copyright (c) Microsoft
# Licensed under the MIT License.
# Written by Bin Xiao (leoxiaobin@gmail.com)
# Modified by Bowen Cheng (bcheng9@illinois.edu)
# ------------------------------------------------------------------------------

from __future__ import absolute_import
from __future__ import division
from __future__ import print_function

from __future__ import absolute_import
from __future__ import division
from __future__ import print_function

import os
import logging

import torch
import torch.nn as nn


BN_MOMENTUM = 0.1
logger = logging.getLogger(__name__)


def conv3x3(in_planes, out_planes, stride=1):
    """3x3 convolution with padding"""
    return nn.Conv2d(in_planes, out_planes, kernel_size=3, stride=stride,
                     padding=1, bias=False)


class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, inplanes, planes, stride=1, downsample=None):
        super(BasicBlock, self).__init__()
        self.conv1 = conv3x3(inplanes, planes, stride)
        self.bn1 = nn.BatchNorm2d(planes, momentum=BN_MOMENTUM)
        self.relu = nn.ReLU(inplace=True)
        self.conv2 = conv3x3(planes, planes)
        self.bn2 = nn.BatchNorm2d(planes, momentum=BN_MOMENTUM)
        self.downsample = downsample
        self.stride = stride

    def forward(self, x):
        residual = x

        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)

        out = self.conv2(out)
        out = self.bn2(out)

        if self.downsample is not None:
            residual = self.downsample(x)

        out += residual
        out = self.relu(out)

        return out

```

```
64
65      class Bottleneck(nn.Module):
66          expansion = 4
67
68          def __init__(self, inplanes, planes, stride=1, downsample=None):
69              super(Bottleneck, self).__init__()
70              self.conv1 = nn.Conv2d(inplanes, planes, kernel_size=1, bias=False)
71              self.bn1 = nn.BatchNorm2d(planes, momentum=BN_MOMENTUM)
72              self.conv2 = nn.Conv2d(planes, planes, kernel_size=3, stride=stride,
73                                     padding=1, bias=False)
74              self.bn2 = nn.BatchNorm2d(planes, momentum=BN_MOMENTUM)
75              self.conv3 = nn.Conv2d(planes, planes * self.expansion, kernel_size=1,
76                                     bias=False)
77              self.bn3 = nn.BatchNorm2d(planes * self.expansion,
78                                        momentum=BN_MOMENTUM)
79              self.relu = nn.ReLU(inplace=True)
80              self.downsample = downsample
81              self.stride = stride
82
83          def forward(self, x):
84              residual = x
85
86              out = self.conv1(x)
87              out = self.bn1(out)
88              out = self.relu(out)
89
90              out = self.conv2(out)
91              out = self.bn2(out)
92              out = self.relu(out)
93
94              out = self.conv3(out)
95              out = self.bn3(out)
96
97              if self.downsample is not None:
98                  residual = self.downsample(x)
99
100             out += residual
101             out = self.relu(out)
102
103             return out
104
105
106     class HighResolutionModule(nn.Module):
107         def __init__(self, num_branches, blocks, num_blocks, num_inchannels,
108                      num_channels, fuse_method, multi_scale_output=True):
109             super(HighResolutionModule, self).__init__()
110             self._check_branches(
111                 num_branches, blocks, num_blocks, num_inchannels, num_channels)
112
113             self.num_inchannels = num_inchannels
114             self.fuse_method = fuse_method
115             self.num_branches = num_branches
116
117             self.multi_scale_output = multi_scale_output
118
119             self.branches = self._make_branches(
120                 num_branches, blocks, num_blocks, num_channels)
121             self.fuse_layers = self._make_fuse_layers()
122             self.relu = nn.ReLU(True)
123
124         def _check_branches(self, num_branches, blocks, num_blocks,
125                             num_inchannels, num_channels):
126             if num_branches != len(num_blocks):
127                 error_msg = 'NUM_BRANCHES({}) <> NUM_BLOCKS({})'.format(
128                     num_branches, len(num_blocks))
```

```
                logger.error(error_msg)
                raise ValueError(error_msg)

            if num_branches != len(num_channels):
                error_msg = 'NUM_BRANCHES({}) <> NUM_CHANNELS({})'.format(
                    num_branches, len(num_channels))
                logger.error(error_msg)
                raise ValueError(error_msg)

            if num_branches != len(num_inchannels):
                error_msg = 'NUM_BRANCHES({}) <> NUM_INCHANNELS({})'.format(
                    num_branches, len(num_inchannels))
                logger.error(error_msg)
                raise ValueError(error_msg)

        def _make_one_branch(self, branch_index, block, num_blocks, num_channels,
                             stride=1):
            downsample = None
            if stride != 1 or \
               self.num_inchannels[branch_index] != num_channels[branch_index] *
block.expansion:
                downsample = nn.Sequential(
                    nn.Conv2d(self.num_inchannels[branch_index],
                              num_channels[branch_index] * block.expansion,
                              kernel_size=1, stride=stride, bias=False),
                    nn.BatchNorm2d(num_channels[branch_index] * block.expansion,
                                   momentum=BN_MOMENTUM),
                )

            layers = []
            layers.append(block(self.num_inchannels[branch_index],
                                num_channels[branch_index], stride, downsample))
            self.num_inchannels[branch_index] = \
                num_channels[branch_index] * block.expansion
            for i in range(1, num_blocks[branch_index]):
                layers.append(block(self.num_inchannels[branch_index],
                                    num_channels[branch_index]))

            return nn.Sequential(*layers)

        def _make_branches(self, num_branches, block, num_blocks, num_channels):
            branches = []

            for i in range(num_branches):
                branches.append(
                    self._make_one_branch(i, block, num_blocks, num_channels))

            return nn.ModuleList(branches)

        def _make_fuse_layers(self):
            if self.num_branches == 1:
                return None

            num_branches = self.num_branches
            num_inchannels = self.num_inchannels
            fuse_layers = []
            for i in range(num_branches if self.multi_scale_output else 1):
                fuse_layer = []
                for j in range(num_branches):
                    if j > i:
                        fuse_layer.append(nn.Sequential(
                            nn.Conv2d(num_inchannels[j],
                                      num_inchannels[i],
                                      1,
                                      1,
```

```
                                    0,
                                    bias=False),
                            nn.BatchNorm2d(num_inchannels[i]),
                            nn.Upsample(scale_factor=2**(j-i), mode='nearest')))
                    elif j == i:
                        fuse_layer.append(None)
                    else:
                        conv3x3s = []
                        for k in range(i-j):
                            if k == i - j - 1:
                                num_outchannels_conv3x3 = num_inchannels[i]
                                conv3x3s.append(nn.Sequential(
                                    nn.Conv2d(num_inchannels[j],
                                              num_outchannels_conv3x3,
                                              3, 2, 1, bias=False),
                                    nn.BatchNorm2d(num_outchannels_conv3x3)))
                            else:
                                num_outchannels_conv3x3 = num_inchannels[j]
                                conv3x3s.append(nn.Sequential(
                                    nn.Conv2d(num_inchannels[j],
                                              num_outchannels_conv3x3,
                                              3, 2, 1, bias=False),
                                    nn.BatchNorm2d(num_outchannels_conv3x3),
                                    nn.ReLU(True)))
                        fuse_layer.append(nn.Sequential(*conv3x3s))
                fuse_layers.append(nn.ModuleList(fuse_layer))

            return nn.ModuleList(fuse_layers)

        def get_num_inchannels(self):
            return self.num_inchannels

        def forward(self, x):
            if self.num_branches == 1:
                return [self.branches[0](x[0])]

            for i in range(self.num_branches):
                x[i] = self.branches[i](x[i])

            x_fuse = []

            for i in range(len(self.fuse_layers)):
                y = x[0] if i == 0 else self.fuse_layers[i][0](x[0])
                for j in range(1, self.num_branches):
                    if i == j:
                        y = y + x[j]
                    else:
                        y = y + self.fuse_layers[i][j](x[j])
                x_fuse.append(self.relu(y))

            return x_fuse


    blocks_dict = {
        'BASIC': BasicBlock,
        'BOTTLENECK': Bottleneck
    }


    # https://github.com/HRNet/HRNet-Image-
Classification/blob/master/lib/models/cls_hrnet.py
    class HRNet(nn.Module):

        def __init__(self, cfg, **kwargs):
            super(HRNet, self).__init__()
```

```
257
258             self._frames_in  = cfg['frames_in']
259             self._frames_out = cfg['frames_out']
260             self._out_scales = cfg['out_scales']
261             self._stem_strides  = cfg['MODEL']['EXTRA']['STEM']['STRIDES']
262             self._stem_inplanes = cfg['MODEL']['EXTRA']['STEM']['INPLANES']
263
264             self.conv1 = nn.Conv2d(3*self._frames_in, self._stem_inplanes, kernel_size=3,
stride=self._stem_strides[0], padding=1, bias=False)
265             self.bn1   = nn.BatchNorm2d(self._stem_inplanes, momentum=BN_MOMENTUM)
266             self.conv2 = nn.Conv2d(self._stem_inplanes, self._stem_inplanes, kernel_size=3,
stride=self._stem_strides[1], padding=1, bias=False)
267             self.bn2   = nn.BatchNorm2d(self._stem_inplanes, momentum=BN_MOMENTUM)
268             self.relu  = nn.ReLU(inplace=True)
269
270             self.stage1_cfg = cfg['MODEL']['EXTRA']['STAGE1']
271             num_channels    = self.stage1_cfg['NUM_CHANNELS'][0]
272             block = blocks_dict[self.stage1_cfg['BLOCK']]
273             num_blocks = self.stage1_cfg['NUM_BLOCKS'][0]
274             self.layer1 = self._make_layer(block, self._stem_inplanes, num_channels,
num_blocks)
275             stage1_out_channel = block.expansion*num_channels
276
277             self.stage2_cfg = cfg['MODEL']['EXTRA']['STAGE2']
278             num_channels = self.stage2_cfg['NUM_CHANNELS']
279             block = blocks_dict[self.stage2_cfg['BLOCK']]
280             num_channels = [
281                 num_channels[i] * block.expansion for i in range(len(num_channels))]
282             self.transition1 = self._make_transition_layer(
283                 [stage1_out_channel], num_channels)
284             self.stage2, pre_stage_channels = self._make_stage(
285                 self.stage2_cfg, num_channels)
286
287             self.stage3_cfg = cfg['MODEL']['EXTRA']['STAGE3']
288             num_channels = self.stage3_cfg['NUM_CHANNELS']
289             block = blocks_dict[self.stage3_cfg['BLOCK']]
290             num_channels = [
291                 num_channels[i] * block.expansion for i in range(len(num_channels))]
292             self.transition2 = self._make_transition_layer(
293                 pre_stage_channels, num_channels)
294             self.stage3, pre_stage_channels = self._make_stage(
295                 self.stage3_cfg, num_channels)
296
297             self.stage4_cfg = cfg['MODEL']['EXTRA']['STAGE4']
298             num_channels = self.stage4_cfg['NUM_CHANNELS']
299             block = blocks_dict[self.stage4_cfg['BLOCK']]
300             num_channels = [
301                 num_channels[i] * block.expansion for i in range(len(num_channels))]
302             self.transition3 = self._make_transition_layer(
303                 pre_stage_channels, num_channels)
304             self.stage4, pre_stage_channels = self._make_stage(self.stage4_cfg,
num_channels, multi_scale_output=True)
305
306             self.num_deconvs       = cfg['MODEL']['EXTRA']['DECONV']['NUM_DECONVS']
307             self.deconv_config     = cfg['MODEL']['EXTRA']['DECONV']
308             self.pretrained_layers = cfg['MODEL']['EXTRA']['PRETRAINED_LAYERS']
309
310             self.deconv_layers = self._make_deconv_layers(cfg, pre_stage_channels[0])
311             self.final_layers  = self._make_final_layers(cfg, pre_stage_channels)
312
313         def _get_deconv_cfg(self, deconv_kernel):
314             if deconv_kernel == 4:
315                 padding = 1
316                 output_padding = 0
317             elif deconv_kernel == 3:
```

```
                padding = 1
                output_padding = 1
            elif deconv_kernel == 2:
                padding = 0
                output_padding = 0

            return deconv_kernel, padding, output_padding

        def _make_final_layers(self, cfg, channels):
            kernel_size = cfg['MODEL']['EXTRA']['FINAL_CONV_KERNEL']
            layers      = []
            for scale in self._out_scales:
                layers.append( nn.Conv2d(in_channels=channels[scale],
out_channels=self._frames_out, kernel_size=kernel_size) )
            return nn.ModuleList(layers)

        def _make_deconv_layers(self, cfg, input_channels):
            extra      = cfg.MODEL.EXTRA
            deconv_cfg = extra.DECONV

            deconv_layers = []
            for i in range(deconv_cfg.NUM_DECONVS):
                output_channels                         = input_channels
                deconv_kernel, padding, output_padding =
self._get_deconv_cfg(deconv_cfg.KERNEL_SIZE[i])

                layers = []
                layers.append(nn.Sequential(
                    nn.ConvTranspose2d(
                        in_channels=input_channels,
                        out_channels=output_channels,
                        kernel_size=deconv_kernel,
                        stride=2,
                        padding=padding,
                        output_padding=output_padding,
                        bias=False),
                    nn.BatchNorm2d(output_channels, momentum=BN_MOMENTUM),
                    nn.ReLU(inplace=True)
                ))
                deconv_layers.append(nn.Sequential(*layers))
                input_channels = output_channels

            return nn.ModuleList(deconv_layers)

        def _make_transition_layer(
                self, num_channels_pre_layer, num_channels_cur_layer):
            num_branches_cur = len(num_channels_cur_layer)
            num_branches_pre = len(num_channels_pre_layer)

            transition_layers = []
            for i in range(num_branches_cur):
                if i < num_branches_pre:
                    if num_channels_cur_layer[i] != num_channels_pre_layer[i]:
                        transition_layers.append(nn.Sequential(
                            nn.Conv2d(num_channels_pre_layer[i],
                                      num_channels_cur_layer[i],
                                      3,
                                      1,
                                      1,
                                      bias=False),
                            nn.BatchNorm2d(
                                num_channels_cur_layer[i], momentum=BN_MOMENTUM),
                            nn.ReLU(inplace=True)))
                    else:
                        transition_layers.append(None)
```

```
                else:
                    conv3x3s = []
                    for j in range(i+1-num_branches_pre):
                        inchannels = num_channels_pre_layer[-1]
                        outchannels = num_channels_cur_layer[i] \
                            if j == i-num_branches_pre else inchannels
                        conv3x3s.append(nn.Sequential(
                            nn.Conv2d(
                                inchannels, outchannels, 3, 2, 1, bias=False),
                            nn.BatchNorm2d(outchannels, momentum=BN_MOMENTUM),
                            nn.ReLU(inplace=True)))
                    transition_layers.append(nn.Sequential(*conv3x3s))

            return nn.ModuleList(transition_layers)

        def _make_layer(self, block, inplanes, planes, blocks, stride=1):
            downsample = None
            if stride != 1 or inplanes != planes * block.expansion:
                downsample = nn.Sequential(
                    nn.Conv2d(inplanes, planes * block.expansion,
                              kernel_size=1, stride=stride, bias=False),
                    nn.BatchNorm2d(planes * block.expansion, momentum=BN_MOMENTUM),
                )

            layers = []
            layers.append(block(inplanes, planes, stride, downsample))
            inplanes = planes * block.expansion
            for i in range(1, blocks):
                layers.append(block(inplanes, planes))

            return nn.Sequential(*layers)

        def _make_stage(self, layer_config, num_inchannels,
                        multi_scale_output=True):
            num_modules = layer_config['NUM_MODULES']
            num_branches = layer_config['NUM_BRANCHES']
            num_blocks = layer_config['NUM_BLOCKS']
            num_channels = layer_config['NUM_CHANNELS']
            block = blocks_dict[layer_config['BLOCK']]
            fuse_method = layer_config['FUSE_METHOD']

            modules = []
            for i in range(num_modules):
                # multi_scale_output is only used last module
                if not multi_scale_output and i == num_modules - 1:
                    reset_multi_scale_output = False
                else:
                    reset_multi_scale_output = True

                modules.append(
                    HighResolutionModule(num_branches,
                                         block,
                                         num_blocks,
                                         num_inchannels,
                                         num_channels,
                                         fuse_method,
                                         reset_multi_scale_output)
                )
                num_inchannels = modules[-1].get_num_inchannels()

            return nn.Sequential(*modules), num_inchannels

        def forward(self, x):
            x = self.conv1(x)
            x = self.bn1(x)
```

```
            x = self.relu(x)
            x = self.conv2(x)
            x = self.bn2(x)
            x = self.relu(x)
            x = self.layer1(x)

            x_list = []
            for i in range(self.stage2_cfg['NUM_BRANCHES']):
                if self.transition1[i] is not None:
                    x_list.append(self.transition1[i](x))
                else:
                    x_list.append(x)
            y_list = self.stage2(x_list)

            x_list = []
            for i in range(self.stage3_cfg['NUM_BRANCHES']):
                if self.transition2[i] is not None:
                    x_list.append(self.transition2[i](y_list[-1]))
                else:
                    x_list.append(y_list[i])
            y_list = self.stage3(x_list)

            x_list = []
            for i in range(self.stage4_cfg['NUM_BRANCHES']):
                if self.transition3[i] is not None:
                    x_list.append(self.transition3[i](y_list[-1]))
                else:
                    x_list.append(y_list[i])
            y_list = self.stage4(x_list)

            y_out = {}
            for scale in self._out_scales:
                x = y_list[scale]
                for i in range(self.num_deconvs):
                    x = self.deconv_layers[i][scale](x)
                y = self.final_layers[scale](x)
                y_out[scale] = y
            return y_out

        def init_weights(self, pretrained='',):
            logger.info('=> init weights from normal distribution')
            for m in self.modules():
                if isinstance(m, nn.Conv2d):
                    nn.init.kaiming_normal_(
                        m.weight, mode='fan_out', nonlinearity='relu')
                elif isinstance(m, nn.BatchNorm2d):
                    nn.init.constant_(m.weight, 1)
                    nn.init.constant_(m.bias, 0)
            if os.path.isfile(pretrained):
                pretrained_dict = torch.load(pretrained)
                logger.info('=> loading pretrained model {}'.format(pretrained))
                model_dict = self.state_dict()
                pretrained_dict = {k: v for k, v in pretrained_dict.items()
                                   if k in model_dict.keys()}
                for k, _ in pretrained_dict.items():
                    logger.info(
                        '=> loading {} pretrained model {}'.format(k, pretrained))
                model_dict.update(pretrained_dict)
                self.load_state_dict(model_dict)

    """
    if __name__=='__main__':

        import yaml
        from attrdict import AttrDict
```

```
        import torch

        config_path = '/root/src/configs/model/higher_hrnet_w32.yaml'
        with open(config_path, 'r') as f:
            cfg = yaml.safe_load(f)
        #print(cfg)
        cfg = AttrDict(cfg)
        #print(cfg)
        #print(cfg.MODEL)
        model = HigherHRNet(cfg)
        print(model)

        frames_in  = 3
        frames_out = 3
        batch_size = 4

        x = torch.rand((batch_size, 3*frames_in, 288, 512))
        print(x.shape)
        y = model(x)
        #print(y[0].shape, y[1].shape)
        print(y.shape)
    """
```

6. user (2026-07-06T20:20:23.408Z)

```
      from .unet2d import TrackNetV2
      from .resunet2d import ChangsTrackNet
      from .monotrack import MonoTrack
      from .hrnet import HRNet
      from .deepball import DeepBall
      from .ballseg import BallSeg

      __factory = {
          'tracknetv2': TrackNetV2,
          'monotrack': MonoTrack,
          'restracknetv2': ChangsTrackNet,
          'hrnet': HRNet,
          'deepball': DeepBall,
          'ballseg': BallSeg
              }

    def build_model(cfg):
          model_name = cfg['model']['name']
          if not model_name in __factory.keys():
              raise KeyError('invalid model: {}'.format(model_name ))
          if model_name=='tracknetv2' or model_name=='resunet2d' or model_name=='monotrack':
              frames_in  = cfg['model']['frames_in']
              frames_out = cfg['model']['frames_in']
              bilinear   = cfg['model']['bilinear']
              halve_channel = cfg['model']['halve_channel']
              model      = __factory[model_name]( frames_in*3, frames_out, bilinear=bilinear,
halve_channel=halve_channel)
          elif model_name=='higher_hrnet' or model_name=='cls_hrnet' or model_name=='hrnet':
              model = __factory[model_name](cfg['model'])
          elif model_name=='restracknetv2':
              frames_in        = cfg['model']['frames_in']
              frames_out       = cfg['model']['frames_out']
              halve_channel    = cfg['model']['halve_channel']
              mode             = cfg['model']['mode']
              neck_channels    = cfg['model']['neck_channels']
              out_mid_channels = cfg['model']['out_mid_channels']
              blocks           = cfg['model']['blocks']
```

```
            channels        = cfg['model']['channels']
            model = __factory[model_name]( frames_in*3,
                                           frames_out,
                                           mode=mode,
                                           halve_channel=halve_channel,
                                           neck_channels=neck_channels,
                                           out_mid_channels=out_mid_channels,
                                           blocks=blocks, channels=channels)
        elif model_name=='deepball':
            frames_in             = cfg['model']['frames_in']
            frames_out            = cfg['model']['frames_out']
            class_out             = cfg['model']['class_out']
            block_channels        = cfg['model']['block_channels']
            block_maxpools        = cfg['model']['block_maxpools']
            first_conv_kernel_size = cfg['model']['first_conv_kernel_size']
            last_conv_kernel_size  = cfg['model']['last_conv_kernel_size']
            first_conv_stride     = cfg['model']['first_conv_stride']
            model = __factory[model_name]( frames_in*3, frames_out*class_out,
                                           block_channels=block_channels,
                                           block_maxpools=block_maxpools,
                                           first_conv_kernel_size=first_conv_kernel_size,
                                           first_conv_stride=first_conv_stride,
                                           last_conv_kernel_size=last_conv_kernel_size)
        elif model_name=='ballseg':
            frames_in     = cfg['model']['frames_in']
            frames_out    = cfg['model']['frames_out']
            scale_factors = cfg['model']['scale_factors']
            backbone      = cfg['model']['backbone']
            model = __factory[model_name]( in_channels=frames_in*3, nclass=frames_out*1,
backbone=backbone, scale_factors=scale_factors)

        return model


```

7. user (2026-07-06T20:20:24.412Z)

```
    import os.path as osp
    import logging
    from collections import defaultdict
    from hydra.core.hydra_config import HydraConfig
    import numpy as np
    from PIL import Image
    import cv2
    import torch
    from torch import nn

    from models import build_model
    from dataloaders import read_image, get_transform, build_img_transforms
    from utils.image import get_affine_transform, affine_transform
    from .postprocessor import TracknetV2Postprocessor
    from .deepball_postprocessor import DeepBallPostprocessor

    log = logging.getLogger(__name__)

    class TracknetV2Detector(object):

        __postprocessor_factory = {
                'tracknetv2': TracknetV2Postprocessor,
                'deepball': DeepBallPostprocessor,
                }

        def __init__(self, cfg, model=None):
            self._frames_in  = cfg['model']['frames_in']
            self._frames_out = cfg['model']['frames_out']
```

```
            self._input_wh   = (cfg['model']['inp_width'], cfg['model']['inp_height'])

            self._scales          = cfg['detector']['postprocessor']['scales']

            self._2d_input = True
            model_name = cfg['model']['name']
            if model_name in ['tracknetv2', 'hrnet', 'monotrack', 'restracknetv2',
'deepball', 'ballseg']:
                pass
            else:
                raise ValueError('unknown model_name : {}'.format(model_name))

            _, self._transform = build_img_transforms(cfg)

            self._device = cfg['runner']['device']
            if self._device!='cuda':
                assert 0, 'device=cpu not supported'
            if not torch.cuda.is_available():
                assert 0, 'GPU NOT available'
            self._gpus  = cfg['runner']['gpus']

            if model is None:
                self._model = build_model(cfg)
                model_path = cfg['detector']['model_path']
                if model_path is None:
                    output_dir = HydraConfig.get().run.dir
                    model_path = osp.join(output_dir, 'best_model.pth.tar')
                    log.info('Checkpoint is not specified, so it is set as the best model in
{}'.format(output_dir))
                    if not osp.exists(model_path):
                        FileNotFoundError('{} not found'.format(model_path))
                checkpoint = torch.load(model_path)
                self._model.load_state_dict(checkpoint['model_state_dict'])
                self._model = self._model.to(self._device)
                self._model = nn.DataParallel(self._model, device_ids=self._gpus)
            else:
                self._model = model

            self._model.eval()

            postprocessor_name = cfg['detector']['postprocessor']['name']
            if not postprocessor_name in self.__postprocessor_factory.keys():
                raise KeyError('invalid dataset: {}'.format(postprocessor_name ))
            self._postprocessor = self.__postprocessor_factory[postprocessor_name](cfg)

        @property
        def frames_in(self):
            return self._frames_in

        @property
        def frames_out(self):
            return self._frames_out

        @property
        def input_wh(self):
            return self._input_wh

        def run_tensor(self, imgs, affine_mats):
            imgs  = imgs.to(self._device)
            preds = self._model(imgs)
            pp_results  = self._postprocessor.run(preds, affine_mats)

            results = {}
            hms_vis = {}
            for bid in sorted(pp_results.keys()):
```

```
                results[bid] = {}
                hms_vis[bid] = {}
                for eid in sorted(pp_results[bid].keys()):
                    results[bid][eid] = []
                    hms_vis[bid][eid] = []
                    for scale in sorted(pp_results[bid][eid].keys()):
                        scores = pp_results[bid][eid][scale]['scores']
                        xys    = pp_results[bid][eid][scale]['xys']
                        for xy, score in zip(xys, scores):
                            results[bid][eid].append({ 'xy': xy, 'score': score, 'scale':
scale})

                        hm    = pp_results[bid][eid][scale]['hm']
                        trans = pp_results[bid][eid][scale]['trans']
                        hms_vis[bid][eid].append({'hm': hm, 'scale': scale, 'trans': trans})

            return results, hms_vis


```

8. user (2026-07-06T20:20:25.917Z)

```
    from collections import defaultdict
    import numpy as np
    import cv2
    import torch

    from utils.utils import _nms, _top1
    from utils.image import get_affine_transform, affine_transform

    class TracknetV2Postprocessor(object):
        def __init__(self, cfg):
            #print(cfg['detector']['postprocessor'])
            self._score_threshold = cfg['detector']['postprocessor']['score_threshold']
            self._model_name      = cfg['model']['name']
            self._scales          = cfg['detector']['postprocessor']['scales']
            self._blob_det_method = cfg['detector']['postprocessor']['blob_det_method']
            self._use_hm_weight   = cfg['detector']['postprocessor']['use_hm_weight']
            #self._xy_comp_method  = cfg['detector']['postprocessor']['xy_comp_method']
            #print(self._score_threshold, self._scales)

            #self._hm_type = cfg['target_generator']['type']
            #self._sigmas  = cfg['target_generator']['sigmas']
            self._sigmas = cfg['dataloader']['heatmap']['sigmas']
            #self._mags    = cfg['target_generator']['mags']
            #self._min_values = cfg['target_generator']['min_values']
            #print(hm_type, sigmas, mags, min_values)

        """
        def _detect_blob_center(self, hm):
            xy   = np.array([0., 0.])
            visi = False
            max_blob_score = -1
            if np.max(hm) > self._score_threshold:
                visi = True
                th, hm_th        = cv2.threshold(hm, self._score_threshold, 1,
cv2.THRESH_BINARY)
                n_labels, labels = cv2.connectedComponents(hm_th.astype(np.uint8))
                for m in range(1,n_labels):
                    ys, xs = np.where( labels==m )
                    score  = xs.shape[0]
                    #print(xs, ys)
                    if score > max_blob_score:
                        max_blob_score = score
                        xy = np.array([np.mean(xs), np.mean(ys)])
```

```
                        #xy = np.array([np.unique(xs).mean(), np.unique(ys).mean()])
            return xy, visi, max_blob_score
        """

        def _detect_blob_concomp(self, hm):
            xys, scores = [], []
            if np.max(hm) > self._score_threshold:
                visi = True
                th, hm_th        = cv2.threshold(hm, self._score_threshold, 1,
cv2.THRESH_BINARY)
                n_labels, labels = cv2.connectedComponents(hm_th.astype(np.uint8))
                for m in range(1,n_labels):
                    ys, xs = np.where( labels==m )
                    ws     = hm[ys, xs]
                    if self._use_hm_weight:
                        score  = ws.sum()
                        x      = np.sum( np.array(xs) * ws ) / np.sum(ws)
                        y      = np.sum( np.array(ys) * ws ) / np.sum(ws)
                    else:
                        score  = ws.shape[0]
                        x      = np.sum( np.array(xs) ) / ws.shape[0]
                        y      = np.sum( np.array(ys) ) / ws.shape[0]
                        #print(xs, ys)
                        #print(score, x, y)
                    xys.append( np.array([x, y]) )
                    scores.append( score)
            return xys, scores

        def _detect_blob_nms(self, hm, sigma):
            xys, scores  = [], []
            hm_ori       = hm.copy()
            hm_h, hm_w   = hm.shape
            map_x, map_y = np.meshgrid(np.linspace(1, hm_w, hm_w), np.linspace(1, hm_h,
hm_h))
            while True:
                cy, cx = np.unravel_index(np.argmax(hm), hm.shape)
                if hm[cy,cx] <= self._score_threshold:
                    break
                #dist_map = ((map_y - cy)**2) + ((map_x - cx)**2)
                dist_map = ((map_y - (cy+1))**2) + ((map_x - (cx+1))**2)
                ys, xs   = np.where( dist_map<=sigma**2)
                ws       = hm_ori[dist_map <= sigma**2]
                if self._use_hm_weight:
                    score  = ws.sum()
                    x      = np.sum( np.array(xs) * ws ) / np.sum(ws)
                    y      = np.sum( np.array(ys) * ws ) / np.sum(ws)
                else:
                    score  = ws.shape[0]
                    x      = np.sum( np.array(xs) ) / ws.shape[0]
                    y      = np.sum( np.array(ys) ) / ws.shape[0]
                    #print(xs, ys)
                    #print(score, x, y)
                xys.append( np.array([x, y]) )
                scores.append( score)
                hm[dist_map<=sigma**2] = 0.
            return xys, scores

        def run(self, preds, affine_mats):
            results = defaultdict(lambda: defaultdict(dict))
            for scale in self._scales:
                preds_       = preds[scale]
                affine_mats_ = affine_mats[scale].cpu().numpy()
                hms_         = preds_.sigmoid_().cpu().numpy()

                b,s,h,w = hms_.shape
```

```
106                 for i in range(b):
107                     for j in range(s):
108                         #print(i,j)
109                         """
110                         if self._xy_comp_method=='center':
111                             assert 0, 'not yet'
112                             #xy_, visi_, blob_size_ = self._detect_blob_center(hms_[i,j])
113                             xy_, visi_, blob_score_ = self._detect_blob_center(hms_[i,j])
114                         elif self._xy_comp_method=='gravity':
115                             #xy_, visi_, blob_size_ = self._detect_blob_gravity(hms_[i,j])
116                             #xy_, visi_, blob_score_ = self._detect_blob_gravity(hms_[i,j])
117                             xys_, scores_ = self._detect_blob_gravity(hms_[i,j])
118                         """
119                         if self._blob_det_method=='concomp':
120                             xys_, scores_ = self._detect_blob_concomp(hms_[i,j])
121                         elif self._blob_det_method=='nms':
122                             xys_, scores_ = self._detect_blob_nms(hms_[i,j],
self._sigmas[scale])
123                         else:
124                             raise ValueError('undefined xy_comp_method:
{}'.format(self._xy_comp_method))
125                         xys_t_ = []
126                         for xy_ in xys_:
127                             xys_t_.append( affine_transform(xy_, affine_mats_[i]))
128                         #results[i][j][scale] = {'xy': xy_, 'visi': visi_, 'blob_size':
blob_size_}
129                         #results[i][j][scale] = {'xy': xy_, 'visi': visi_, 'blob_score':
blob_score_}
130                         results[i][j][scale] = {'xys': xys_t_, 'scores': scores_, 'hm':
hms_[i,j], 'trans': affine_mats_[i]}
131
132             #print(results)
133             return results
134
135
```

9. user (2026-07-06T20:20:26.490Z)

```
1       from .detector import TracknetV2Detector
2       from .deepball_detector import DeepBallDetector
3
4       __factory = {
5           'tracknetv2' : TracknetV2Detector,
6           'deepball': DeepBallDetector
7           }
8
9       def build_detector(cfg, model=None):
10          detector_name = cfg['detector']['name']
11          if not detector_name in __factory.keys():
12              raise KeyError('invalid detector: {}'.format(detector_name ))
13          return __factory[detector_name](cfg, model=model)
14
15
```

10. assistant (2026-07-06T20:20:32.465Z)

Key facts gathered. The detector uses affine transform (not simple resize) for preprocessing, applies `sigmoid` to heatmaps in postprocessor, and postprocessor picks position from heatmap channels. Now let me read the configs and dataloaders to nail down the exact preprocessing constants and which channel is used.

11. user (2026-07-06T20:20:35.054Z)

```
1       name: hrnet
```

```
    frames_in: 3
    frames_out: 3
    inp_height: 288
    inp_width: 512
    out_height: 288
    out_width: 512
    rgb_diff: False
    out_scales: [0]
    MODEL:
      EXTRA:
        FINAL_CONV_KERNEL: 1
        PRETRAINED_LAYERS: ['*']
        STEM:
          INPLANES: 64
          STRIDES: [1,1]
        STAGE1:
          NUM_MODULES: 1
          NUM_BRANCHES: 1
          BLOCK: BOTTLENECK
          NUM_BLOCKS: [1]
          NUM_CHANNELS: [32]
          FUSE_METHOD: SUM
        STAGE2:
          NUM_MODULES: 1
          NUM_BRANCHES: 2
          BLOCK: BASIC
          NUM_BLOCKS: [2,2]
          NUM_CHANNELS: [16,32]
          FUSE_METHOD: SUM
        STAGE3:
          NUM_MODULES: 1
          NUM_BRANCHES: 3
          BLOCK: BASIC
          NUM_BLOCKS: [2,2,2]
          NUM_CHANNELS: [16,32,64]
          FUSE_METHOD: SUM
        STAGE4:
          NUM_MODULES: 1
          NUM_BRANCHES: 4
          BLOCK: BASIC
          NUM_BLOCKS: [2,2,2,2]
          NUM_CHANNELS: [16,32,64,128]
          FUSE_METHOD: SUM
        DECONV:
          NUM_DECONVS: 0
          KERNEL_SIZE: []
          NUM_BASIC_BLOCKS: 2
      INIT_WEIGHTS: True


```

12. user (2026-07-06T20:20:36.056Z)

```
    name: badminton
    root_dir: /root/datasets/badminton
    video_dirname: video
    frame_dirname: frame
    csv_dirname: csv
    train:
      matches : [ 'match1',
                  'match2',
                  'match3',
                  'match4',
                  'match5',
                  'match6',
```

```
                    'match7',
                    'match8',
                    'match9',
                    'match10',
                    'match11',
                    'match12',
                    'match13',
                    'match14',
                    'match15',
                    'match16',
                    'match17',
                    'match18',
                    'match19',
                    'match20',
                    'match21',
                    'match22',
                    'match23',
                    'match24',
                    'match25',
                    'match26']
        num_clip_ratio: 1.0
      test:
        matches: ['test_match1',
                  'test_match2',
                  'test_match3']
        num_clip_ratio: 1.0

```

13. user (2026-07-06T20:20:39.326Z)

```
total 40
drwxr-xr-x 1 avidu 197609    0 Jul  7 01:49 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 01:49 ..
-rw-r--r-- 1 avidu 197609 5582 Jul  7 01:49 __init__.py
-rw-r--r-- 1 avidu 197609 7957 Jul  7 01:49 dataset_loader.py
drwxr-xr-x 1 avidu 197609    0 Jul  7 01:49 heatmaps
-rw-r--r-- 1 avidu 197609 4145 Jul  7 01:49 img_transforms.py
drwxr-xr-x 1 avidu 197609    0 Jul  7 01:49 samplers
-rw-r--r-- 1 avidu 197609 4388 Jul  7 01:49 seq_transforms.py
---CONFIGS---
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/dataloader/default.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/dataset/badminton.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/dataset/basketball.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/dataset/soccer.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/dataset/tennis.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/dataset/volleyball.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/detector/deepball.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/detector/tracknetv2.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
```

indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/eval.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/extract_frame.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/loss/hm_bce.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/loss/hm_combo.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/loss/hm_focal.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/loss/hm_mse.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/loss/hm_qfl.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/loss/hm_wbce.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/loss/hm_wbce_s2.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/loss/hm_wbce_s3.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/loss/seg_ssd.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/model/ballseg.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/model/deepball.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/model/deepball_large.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/model/monotrack.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/model/restracknetv2.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/model/tracknetv2.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/model/wasb.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/optimizer/adadelta_constant.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/optimizer/adadelta_multistep.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/optimizer/adam_multistep.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/runner/eval.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/runner/extract_frame.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/tracker/intra_frame_peak.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/tracker/online.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-
SBDT/src/configs/transform/color.yaml

C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/transform/crop.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/transform/default.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/transform/full.yaml
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/configs/transform/hflip.yaml

14. assistant (2026-07-06T20:20:43.061Z)

Now the critical preprocessing files. Let me read the dataloaders `__init__.py`, `img_transforms.py`, `dataset_loader.py`, and the utils/image + utils/utils, plus the detector config for how `run_tensor` is fed.

15. user (2026-07-06T20:20:44.113Z)

```
1       import os
2       import os.path as osp
3       from omegaconf import DictConfig
4       from torch.utils.data import DataLoader
5
6       import dataloaders.img_transforms as T
7       import dataloaders.seq_transforms as ST
8
9       from .dataset_loader import ImageDataset, read_image, get_transform
10      from .samplers import select_sampler, RandomSampler
11      from datasets import select_dataset
12
13      def build_img_transforms(cfg):
14          transform_train = T.Compose([
15              T.ToTensor(),
16              T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
17          ])
18          transform_test = T.Compose([
19              T.ToTensor(),
20              T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
21          ])
22          return transform_train, transform_test
23
24      def build_seq_transforms(cfg):
25          transform_train = ST.SeqTransformCompose([
26              ST.RandomHorizontalFlipping(cfg['transform']['train']['horizontal_flip']['p']),
27              ST.RandomCropping(p=cfg['transform']['train']['crop']['p'],
max_rescale=cfg['transform']['train']['crop']['max_rescale']),
28          ])
29          transform_test = None
30          return transform_train, transform_test
31
32      def build_dataloader(
33          cfg: DictConfig,
34      ):
35          dataset = select_dataset(cfg)
36
37          transform_train, transform_test         = build_img_transforms(cfg)
38          seq_transform_train, seq_transform_test = build_seq_transforms(cfg)
39
40          input_wh  = (cfg['model']['inp_width'], cfg['model']['inp_height'])
41          output_wh = (cfg['model']['out_width'], cfg['model']['out_height'])
42
43          train_sampler, test_sampler, train_clip_samplers, test_clip_samplers =
```

```
select_sampler(cfg['dataloader']['sampler'], dataset)

        model_name = cfg['model']['name']

        fp1_fname = cfg['runner']['fp1_filename']
        if fp1_fname is not None:
            fp1_fpath = osp.join(cfg['output_dir'], fp1_fname)
        else:
            fp1_fpath = None

        train_clip_datasets = {}
        test_clip_datasets  = {}
        if model_name in ['tracknetv2', 'hrnet', 'monotrack', 'restracknetv2', 'deepball',
'ballseg']:
            train_dataset = ImageDataset(cfg,
                                         dataset=dataset.train,
                                         input_wh=input_wh,
                                         output_wh=output_wh,
                                         transform=transform_train,
                                         seq_transform=seq_transform_train,
                                         fp1_fpath=fp1_fpath,
                                         )
            test_dataset  = ImageDataset(cfg,
                                         dataset=dataset.test,
                                         input_wh=input_wh,
                                         output_wh=output_wh,
                                         transform=transform_test,
                                         seq_transform=seq_transform_test,
                                         is_train=False,
                                         )

            for key, clip in dataset.train_clips.items():
                clip_dataset  = ImageDataset(cfg,
                                             dataset=clip,
                                             input_wh=input_wh,
                                             output_wh=output_wh,
                                             transform=transform_test,
                                             is_train=False,
                                             )
                train_clip_datasets[key] = clip_dataset

            for key, clip in dataset.test_clips.items():
                clip_dataset  = ImageDataset(cfg,
                                             dataset=clip,
                                             input_wh=input_wh,
                                             output_wh=output_wh,
                                             transform=transform_test,
                                             is_train=False,
                                             )
                test_clip_datasets[key] = clip_dataset
        else:
            raise ValueError('unknwon model_name : {}'.format(model_name))


        train_loader = DataLoader(dataset=train_dataset,
                                  batch_sampler=train_sampler,
                                  num_workers=cfg['dataloader']['train_num_workers'],
                                  pin_memory=False)
        test_loader = DataLoader(dataset=test_dataset,
                                 batch_sampler=test_sampler,
                                 num_workers=cfg['dataloader']['test_num_workers'],
                                 pin_memory=False)

        train_clip_loaders_and_gts = {}
        for key, clip_dataset in train_clip_datasets.items():
```

```
            clip_loader = DataLoader(dataset=clip_dataset,
                                     batch_sampler=train_clip_samplers[key],
num_workers=cfg['dataloader']['inference_video_num_workers'],
                                     pin_memory=False)
            train_clip_loaders_and_gts[key] = {'clip_loader': clip_loader, 'clip_gt':
dataset.train_clip_gts[key]}

        test_clip_loaders_and_gts = {}
        for key, clip_dataset in test_clip_datasets.items():
            clip_loader = DataLoader(dataset=clip_dataset,
                                     batch_sampler=test_clip_samplers[key],
num_workers=cfg['dataloader']['inference_video_num_workers'],
                                     pin_memory=False)
            test_clip_loaders_and_gts[key] = {'clip_loader': clip_loader, 'clip_gt':
dataset.test_clip_gts[key]}

        return train_loader, test_loader, train_clip_loaders_and_gts,
test_clip_loaders_and_gts


```

16. user (2026-07-06T20:20:45.292Z)

```
      from torchvision.transforms import *
      from PIL import Image
      import random
      import math

      class ResizeWithEqualScale(object):
          """
          Resize an image with equal scale as the original image.
          Args:
              height (int): resized height.
              width (int): resized width.
              interpolation: interpolation manner.
              fill_color (tuple): color for padding.
          """
          def __init__(self, height, width, interpolation=Image.BILINEAR, fill_color=(0,0,0)):
              self.height = height
              self.width = width
              self.interpolation = interpolation
              self.fill_color = fill_color

          def __call__(self, img):
              width, height = img.size
              if self.height / self.width >= height / width:
                  height = int(self.width * (height / width))
                  width = self.width
              else:
                  width = int(self.height * (width / height))
                  height = self.height

              resized_img = img.resize((width, height), self.interpolation)
              new_img = Image.new('RGB', (self.width, self.height), self.fill_color)
              new_img.paste(resized_img, (int((self.width - width) / 2), int((self.height -
height) / 2)))

              return new_img


      class RandomCroping(object):
          """
          With a probability, first increase image size to (1 + 1/8), and then perform random
```

```
crop.
        Args:
            p (float): probability of performing this transformation. Default: 0.5.
        """
        def __init__(self, p=0.5, interpolation=Image.BILINEAR):
            self.p = p
            self.interpolation = interpolation

        def __call__(self, img):
            """
            Args:
                img (PIL Image): Image to be cropped.
            Returns:
                PIL Image: Cropped image.
            """
            width, height = img.size
            if random.uniform(0, 1) >= self.p:
                return img

            new_width, new_height = int(round(width * 1.125)), int(round(height * 1.125))
            resized_img = img.resize((new_width, new_height), self.interpolation)
            x_maxrange = new_width - width
            y_maxrange = new_height - height
            x1 = int(round(random.uniform(0, x_maxrange)))
            y1 = int(round(random.uniform(0, y_maxrange)))
            croped_img = resized_img.crop((x1, y1, x1 + width, y1 + height))

            return croped_img


    class RandomErasing(object):
        """
        Randomly selects a rectangle region in an image and erases its pixels.
        Reference:
            Zhong et al. Random Erasing Data Augmentation. arxiv: 1708.04896, 2017.
        Args:
            probability: The probability that the Random Erasing operation will be
performed.
            sl: Minimum proportion of erased area against input image.
            sh: Maximum proportion of erased area against input image.
            r1: Minimum aspect ratio of erased area.
            mean: Erasing value.
        """

        def __init__(self, probability = 0.5, sl = 0.02, sh = 0.4, r1 = 0.3, mean=[0.4914,
0.4822, 0.4465]):
            self.probability = probability
            self.mean = mean
            self.sl = sl
            self.sh = sh
            self.r1 = r1

        def __call__(self, img):

            if random.uniform(0, 1) >= self.probability:
                return img

            for attempt in range(100):
                area = img.size()[1] * img.size()[2]

                target_area = random.uniform(self.sl, self.sh) * area
                aspect_ratio = random.uniform(self.r1, 1/self.r1)

                h = int(round(math.sqrt(target_area * aspect_ratio)))
                w = int(round(math.sqrt(target_area / aspect_ratio)))
```

```

                if w < img.size()[2] and h < img.size()[1]:
                    x1 = random.randint(0, img.size()[1] - h)
                    y1 = random.randint(0, img.size()[2] - w)
                    if img.size()[0] == 3:
                        img[0, x1:x1+h, y1:y1+w] = self.mean[0]
                        img[1, x1:x1+h, y1:y1+w] = self.mean[1]
                        img[2, x1:x1+h, y1:y1+w] = self.mean[2]
                    else:
                        img[0, x1:x1+h, y1:y1+w] = self.mean[0]
                    return img

            return img


```

17. user (2026-07-06T20:20:46.145Z)

```
import os.path as osp
import json
import logging
import random
from collections import defaultdict
import numpy as np
import functools
from PIL import Image
import cv2
import torch
from torch.utils.data import Dataset
import torchvision.transforms.functional as TF

from utils import read_image
from utils.image import get_affine_transform, affine_transform
from .heatmaps import select_heatmap_generator

log = logging.getLogger(__name__)

def get_transform(img, input_wh, inv=0):
    h,w,_ = img.shape
    c     = np.array([w / 2., h / 2.], dtype=np.float32)
    s     = max(h, w) * 1.0
    input_w, input_h = input_wh
    trans = get_affine_transform(c, s, 0, [input_w, input_h], inv=inv)
    return trans

def get_color_jitter_factors(brightness, contrast, saturation, hue):
    brightness_factor = np.random.uniform(max(0,1-brightness), 1+brightness)
    contrast_factor   = np.random.uniform(max(0,1-contrast), 1+contrast)
    saturation_factor = np.random.uniform(max(0,1-saturation), 1+saturation)
    hue_factor        = np.random.uniform(-hue, hue)
    return brightness_factor, contrast_factor, saturation_factor, hue_factor

class ImageDataset(Dataset):
    def __init__(self,
                 cfg,
                 dataset,
                 input_wh,
                 output_wh=None,
                 transform=None,
                 seq_transform=None,
                 is_train=True,
                 fp1_fpath=None,
    ):
        self._dataset       = dataset
        self._transform     = transform
```

```
48              self._seq_transform = seq_transform
49              self._input_wh      = input_wh
50              self._output_wh     = input_wh if output_wh is None else output_wh
51
52              self._fp1_fpath = fp1_fpath
53
54              self._hm_generator = select_heatmap_generator(cfg['dataloader']['heatmap'])
55
56              self._is_train      = is_train
57
58              if is_train:
59                  self._color_jitter_p          =
cfg['transform']['train']['color_jitter']['p']
60                  self._color_jitter_brightness =
cfg['transform']['train']['color_jitter']['brightness']
61                  self._color_jitter_contrast   =
cfg['transform']['train']['color_jitter']['contrast']
62                  self._color_jitter_saturation =
cfg['transform']['train']['color_jitter']['saturation']
63                  self._color_jitter_hue        =
cfg['transform']['train']['color_jitter']['hue']
64              else:
65                  self._color_jitter_p          =
cfg['transform']['test']['color_jitter']['p']
66                  self._color_jitter_brightness =
cfg['transform']['test']['color_jitter']['brightness']
67                  self._color_jitter_contrast   =
cfg['transform']['test']['color_jitter']['contrast']
68                  self._color_jitter_saturation =
cfg['transform']['test']['color_jitter']['saturation']
69                  self._color_jitter_hue        =
cfg['transform']['test']['color_jitter']['hue']
70
71              self._rgb_diff   = cfg['model']['rgb_diff']
72              self._frames_in  = cfg['model']['frames_in']
73              self._out_scales = cfg['model']['out_scales']
74              if len(self._out_scales)!=1:
75                  assert 0
76              if self._rgb_diff and self._frames_in!=2:
77                  raise ValueError('rgb_diff=True supported only with frames_in=2')
78
79          def __len__(self):
80              return len(self._dataset)
81
82          def __getitem__(self, index):
83
84              fp1_im_list = []
85              if self._fp1_fpath is not None:
86                  if osp.exists(self._fp1_fpath):
87                      with open(self._fp1_fpath, 'r') as f:
88                          data = f.read().split('\n')
89                          fp1_im_list = data[:-1]
90
91              img_paths = self._dataset[index]['frames']
92              annos     = self._dataset[index]['annos']
93
94              img_paths_out = []
95              for anno in annos:
96                  img_paths_out.append( anno['frame_path'])
97
98              input_w, input_h   = self._input_wh
99              output_w, output_h = self._output_wh
100
101             trans_input      = None
102             trans_input_inv  = None
```

```
            trans_outputs     = defaultdict(list)
            trans_outputs_inv = defaultdict(list)

            apply_color_jitter = True
            if random.uniform(0,1) >= self._color_jitter_p:
                apply_color_jitter = False
            else:
                brightness_factor, contrast_factor, saturation_factor, hue_factor =
get_color_jitter_factors(self._color_jitter_brightness, self._color_jitter_contrast,
self._color_jitter_saturation, self._color_jitter_hue)

            imgs, xys, visis = [], [], []
            hms = defaultdict(list)

            for idx, img_path in enumerate(img_paths):
                img = read_image(img_path)

                if trans_input is None:
                    trans_input  = get_transform(np.asarray(img), self._input_wh)
                    out_w, out_h = self._output_wh
                    for scale in self._out_scales:
                        trans_outputs[scale] = get_transform(np.asarray(img), (out_w,
out_h))
                        out_w = out_w // 2
                        out_h = out_h // 2

                    if not self._is_train:
                        trans_input_inv = get_transform(np.asarray(img), self._input_wh,
inv=1)
                        out_w, out_h    = self._output_wh
                        for scale in self._out_scales:
                            trans_outputs_inv[scale] = get_transform(np.asarray(img),
(out_w, out_h), inv=1)
                            out_w = out_w // 2
                            out_h = out_h // 2

                img = Image.fromarray( cv2.warpAffine(np.array(img), trans_input,
self._input_wh, flags=cv2.INTER_LINEAR) )
                imgs.append(img)

            # hms (targets)
            for idx, (img_path, anno) in enumerate(zip(img_paths, annos)):
                binary = True
                if img_path in fp1_im_list:
                    binary = False

                px, py = anno['center'].xy
                visi   = anno['center'].is_visible

                xys.append([px, py])
                visis.append(visi)
                out_w, out_h = self._output_wh
                for scale in self._out_scales:
                    if visi:
                        ct     = affine_transform(np.array([px,py]), trans_outputs[scale])
                        ct_int = ct.astype(np.int32)
                        hm     = self._hm_generator((out_w,out_h), ct_int, binary=binary)
                    else:
                        hm     = self._hm_generator((out_w,out_h), (-1.,-1.))

                    hm = np.expand_dims(hm, axis=0)
                    hms[scale].append(hm)
                    out_w = out_w // 2
                    out_h = out_h // 2

```

```
            imgs_t = []
            hms_t  = defaultdict(list)
            for img in imgs:
                # color gitter (data augmentation)
                if apply_color_jitter:
                    img = TF.adjust_brightness(img, brightness_factor)
                    img = TF.adjust_contrast(img, contrast_factor)
                    img = TF.adjust_saturation(img, saturation_factor)
                    img = TF.adjust_hue(img, hue_factor)
                img_t = self._transform(img)
                imgs_t.append(img_t)
            for scale in self._out_scales:
                for hm in hms[scale]:
                    hm_t = torch.tensor(hm)
                    hms_t[scale].append(hm_t)

            if self._rgb_diff:
                if len(imgs_t)!=2:
                    raise ValueError('assume 2 images are input but {}
given'.format(len(imgs_t)))
                imgs_t[0] = torch.abs(imgs_t[1] - imgs_t[0])

            imgs_t = torch.cat(imgs_t, dim=0)
            for scale in self._out_scales:
                hms_t[scale] = torch.cat(hms_t[scale], dim=0)
            if self._seq_transform is not None:
                imgs_t, hms_t = self._seq_transform(imgs_t, hms_t)
            xys   = torch.tensor(xys)
            visis = torch.tensor(visis)
            if self._is_train:
                return imgs_t, hms_t
            else:
                return imgs_t, hms_t, trans_outputs_inv, xys, visis, img_paths_out


```

18. assistant (2026-07-06T20:20:54.804Z)

Now I have the full preprocessing chain. Critical details confirmed:
- **ToTensor** (`torchvision.transforms.ToTensor`) ? divides by 255 and converts HWC?CHW, RGB order (from PIL).
- **Normalize** with ImageNet mean `[0.485, 0.456, 0.406]` std `[0.229, 0.224, 0.225]`.
- Preprocessing uses **affine warp** (`get_affine_transform` with center/scale, `cv2.warpAffine`, `INTER_LINEAR`), not a plain resize ? this letterboxes with aspect-ratio preservation (scale = max(h,w)). This matters for parity.
- **read_image** returns PIL RGB.
- Postprocessor applies `sigmoid` then per-channel blob detection; the tracker picks the frame.
Let me confirm `get_affine_transform`, `read_image`, `_nms`/`_top1`, the detector config, and how the tracker selects which output channel maps to the "current" frame.

19. user (2026-07-06T20:20:57.881Z)

```
      # ------------------------------------------------------------------------------
      # Copyright (c) Microsoft
      # Licensed under the MIT License.
      # Written by Bin Xiao (Bin.Xiao@microsoft.com)
      # Modified by Xingyi Zhou
      # ------------------------------------------------------------------------------

      from __future__ import absolute_import
      from __future__ import division
      from __future__ import print_function

      from typing import List, Tuple
```

```
    import numpy as np
    import cv2
    import random

    def flip(img):
      return img[:, :, ::-1].copy()

    def transform_preds(coords, center, scale, output_size):
        target_coords = np.zeros(coords.shape)
        trans = get_affine_transform(center, scale, 0, output_size, inv=1)
        for p in range(coords.shape[0]):
            target_coords[p, 0:2] = affine_transform(coords[p, 0:2], trans)
        return target_coords


    def get_affine_transform(center,
                             scale,
                             rot,
                             output_size,
                             shift=np.array([0, 0], dtype=np.float32),
                             inv=0):
        if not isinstance(scale, np.ndarray) and not isinstance(scale, list):
            scale = np.array([scale, scale], dtype=np.float32)

        scale_tmp = scale
        src_w = scale_tmp[0]
        dst_w = output_size[0]
        dst_h = output_size[1]

        rot_rad = np.pi * rot / 180
        src_dir = get_dir([0, src_w * -0.5], rot_rad)
        dst_dir = np.array([0, dst_w * -0.5], np.float32)

        src = np.zeros((3, 2), dtype=np.float32)
        dst = np.zeros((3, 2), dtype=np.float32)
        src[0, :] = center + scale_tmp * shift
        src[1, :] = center + src_dir + scale_tmp * shift
        dst[0, :] = [dst_w * 0.5, dst_h * 0.5]
        dst[1, :] = np.array([dst_w * 0.5, dst_h * 0.5], np.float32) + dst_dir

        src[2:, :] = get_3rd_point(src[0, :], src[1, :])
        dst[2:, :] = get_3rd_point(dst[0, :], dst[1, :])

        if inv:
            trans = cv2.getAffineTransform(np.float32(dst), np.float32(src))
        else:
            trans = cv2.getAffineTransform(np.float32(src), np.float32(dst))

        return trans


    """
    def affine_transform(pt, t):
        print(pt,pt[0],pt[1])
        new_pt = np.array([pt[0], pt[1], 1.], dtype=np.float32).T
        print(new_pt)
        new_pt = np.dot(t, new_pt)
        print(new_pt)
        return new_pt[:2]
    """
    def affine_transform(pt, t):
        #print(pt,pt[0],pt[1])
        #print(isinstance(pt, Tuple[float, float]))
        #print(isinstance(pt, tuple))
        #print(isinstance(pt, np.ndarray))
```

```
        if not ( isinstance(pt, np.ndarray) and pt.shape==(2,) ):
            raise RuntimeError('pt must be np.ndarray and pt.shape=(2,)')
        #print(t, t.shape)
        if not ( isinstance(t, np.ndarray) and t.shape==(2,3) ):
            raise RuntimeError('t must be np.ndarray and t.shape=(2,3)')
        new_pt = np.array([pt[0], pt[1], 1.], dtype=np.float32).T
        #print(new_pt)
        new_pt = np.dot(t, new_pt)
        #print(new_pt)
        return new_pt[:2]

    def get_3rd_point(a, b):
        direct = a - b
        return b + np.array([-direct[1], direct[0]], dtype=np.float32)


    def get_dir(src_point, rot_rad):
        sn, cs = np.sin(rot_rad), np.cos(rot_rad)

        src_result = [0, 0]
        src_result[0] = src_point[0] * cs - src_point[1] * sn
        src_result[1] = src_point[0] * sn + src_point[1] * cs

        return src_result


    def crop(img, center, scale, output_size, rot=0):
        trans = get_affine_transform(center, scale, rot, output_size)

        dst_img = cv2.warpAffine(img,
                                 trans,
                                 (int(output_size[0]), int(output_size[1])),
                                 flags=cv2.INTER_LINEAR)

        return dst_img


    def gaussian_radius(det_size, min_overlap=0.7):
      height, width = det_size

      a1  = 1
      b1  = (height + width)
      c1  = width * height * (1 - min_overlap) / (1 + min_overlap)
      sq1 = np.sqrt(b1 ** 2 - 4 * a1 * c1)
      r1  = (b1 + sq1) / 2

      a2  = 4
      b2  = 2 * (height + width)
      c2  = (1 - min_overlap) * width * height
      sq2 = np.sqrt(b2 ** 2 - 4 * a2 * c2)
      r2  = (b2 + sq2) / 2

      a3  = 4 * min_overlap
      b3  = -2 * min_overlap * (height + width)
      c3  = (min_overlap - 1) * width * height
      sq3 = np.sqrt(b3 ** 2 - 4 * a3 * c3)
      r3  = (b3 + sq3) / 2
      return min(r1, r2, r3)


    def gaussian2D(shape, sigma=1):
        m, n = [(ss - 1.) / 2. for ss in shape]
        y, x = np.ogrid[-m:m+1,-n:n+1]

        h = np.exp(-(x * x + y * y) / (2 * sigma * sigma))
```

```
    h[h < np.finfo(h.dtype).eps * h.max()] = 0
    return h

def draw_umich_gaussian(heatmap, center, radius, k=1):
  diameter = 2 * radius + 1
  gaussian = gaussian2D((diameter, diameter), sigma=diameter / 6)

  x, y = int(center[0]), int(center[1])

  height, width = heatmap.shape[0:2]

  left, right = min(x, radius), min(width - x, radius + 1)
  top, bottom = min(y, radius), min(height - y, radius + 1)

  masked_heatmap  = heatmap[y - top:y + bottom, x - left:x + right]
  masked_gaussian = gaussian[radius - top:radius + bottom, radius - left:radius + right]
  if min(masked_gaussian.shape) > 0 and min(masked_heatmap.shape) > 0: # TODO debug
    np.maximum(masked_heatmap, masked_gaussian * k, out=masked_heatmap)
  return heatmap

def draw_dense_reg(regmap, heatmap, center, value, radius, is_offset=False):
  diameter = 2 * radius + 1
  gaussian = gaussian2D((diameter, diameter), sigma=diameter / 6)
  value = np.array(value, dtype=np.float32).reshape(-1, 1, 1)
  dim = value.shape[0]
  reg = np.ones((dim, diameter*2+1, diameter*2+1), dtype=np.float32) * value
  if is_offset and dim == 2:
    delta = np.arange(diameter*2+1) - radius
    reg[0] = reg[0] - delta.reshape(1, -1)
    reg[1] = reg[1] - delta.reshape(-1, 1)

  x, y = int(center[0]), int(center[1])

  height, width = heatmap.shape[0:2]

  left, right = min(x, radius), min(width - x, radius + 1)
  top, bottom = min(y, radius), min(height - y, radius + 1)

  masked_heatmap = heatmap[y - top:y + bottom, x - left:x + right]
  masked_regmap = regmap[:, y - top:y + bottom, x - left:x + right]
  masked_gaussian = gaussian[radius - top:radius + bottom,
                             radius - left:radius + right]
  masked_reg = reg[:, radius - top:radius + bottom,
                      radius - left:radius + right]
  if min(masked_gaussian.shape) > 0 and min(masked_heatmap.shape) > 0: # TODO debug
    idx = (masked_gaussian >= masked_heatmap).reshape(
      1, masked_gaussian.shape[0], masked_gaussian.shape[1])
    masked_regmap = (1-idx) * masked_regmap + idx * masked_reg
  regmap[:, y - top:y + bottom, x - left:x + right] = masked_regmap
  return regmap


def draw_msra_gaussian(heatmap, center, sigma):
  tmp_size = sigma * 3
  mu_x = int(center[0] + 0.5)
  mu_y = int(center[1] + 0.5)
  w, h = heatmap.shape[0], heatmap.shape[1]
  ul = [int(mu_x - tmp_size), int(mu_y - tmp_size)]
  br = [int(mu_x + tmp_size + 1), int(mu_y + tmp_size + 1)]
  if ul[0] >= h or ul[1] >= w or br[0] < 0 or br[1] < 0:
    return heatmap
  size = 2 * tmp_size + 1
  x = np.arange(0, size, 1, np.float32)
  y = x[:, np.newaxis]
  x0 = y0 = size // 2
```

```
      g = np.exp(- ((x - x0) ** 2 + (y - y0) ** 2) / (2 * sigma ** 2))
      g_x = max(0, -ul[0]), min(br[0], h) - ul[0]
      g_y = max(0, -ul[1]), min(br[1], w) - ul[1]
      img_x = max(0, ul[0]), min(br[0], h)
      img_y = max(0, ul[1]), min(br[1], w)
      heatmap[img_y[0]:img_y[1], img_x[0]:img_x[1]] = np.maximum(
        heatmap[img_y[0]:img_y[1], img_x[0]:img_x[1]],
        g[g_y[0]:g_y[1], g_x[0]:g_x[1]])
      return heatmap

    def grayscale(image):
        return cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    def lighting_(data_rng, image, alphastd, eigval, eigvec):
        alpha = data_rng.normal(scale=alphastd, size=(3, ))
        image += np.dot(eigvec, eigval * alpha)

    def blend_(alpha, image1, image2):
        image1 *= alpha
        image2 *= (1 - alpha)
        image1 += image2

    def saturation_(data_rng, image, gs, gs_mean, var):
        alpha = 1. + data_rng.uniform(low=-var, high=var)
        blend_(alpha, image, gs[:, :, None])

    def brightness_(data_rng, image, gs, gs_mean, var):
        alpha = 1. + data_rng.uniform(low=-var, high=var)
        image *= alpha

    def contrast_(data_rng, image, gs, gs_mean, var):
        alpha = 1. + data_rng.uniform(low=-var, high=var)
        blend_(alpha, image, gs_mean)

    def color_aug(data_rng, image, eig_val, eig_vec):
        functions = [brightness_, contrast_, saturation_]
        random.shuffle(functions)

        gs = grayscale(image)
        gs_mean = gs.mean()
        for f in functions:
            f(data_rng, image, gs, gs_mean, 0.4)
        lighting_(data_rng, image, 0.1, eig_val, eig_vec)

```

20. user (2026-07-06T20:21:02.648Z)

```
defaults:
        - _self_
name: tracknetv2
model_path: # None
step: 3
postprocessor:
  name: tracknetv2
  score_threshold: 0.5
  scales: [0]
  blob_det_method: concomp
  use_hm_weight: True

===EVAL===
defaults:
        - _self_
        - runner: eval
        - dataset: badminton
        - model: wasb
```

```
        - dataloader: default
        - detector: tracknetv2
        - transform: default
        - tracker: online
hydra:
  run:
    dir: /root/outputs/${hydra.job.name}/${now:%Y-%m-%d_%H-%M-%S}
output_dir:
seed: 1234

===DL_DEFAULT===
train: False
test: False
train_clip: False
test_clip: True
#inference_video: True
sampler:
        name: random
        train_batch_size: 8
        train_shuffle_batch: True
        train_drop_last: True
        test_batch_size: 8
        test_shuffle_batch: False
        test_drop_last: False
        inference_video_batch_size: 8
        inference_video_shuffle_batch: False
        inference_video_drop_last: False
train_num_workers: 8
test_num_workers: 8
inference_video_num_workers: 8
heatmap:
        name: binary_fixed_size
        sigmas: [2.5]
        mags: [1.0]
        min_value: 0.6
```

21. user (2026-07-06T20:21:02.764Z)

```
1       from .intra_frame_peak import IntraFramePeakTracker
2       from .online import OnlineTracker
3
4       __tracker_factory = {
5           'intra_frame_peak': IntraFramePeakTracker,
6           'online': OnlineTracker,
7               }
8
9       def build_tracker(cfg):
10          tracker_name = cfg['tracker']['name']
11          if tracker_name not in __tracker_factory.keys():
12              raise KeyError('unknown tracker: {}'.format(tracker_name))
13          return __tracker_factory[tracker_name](cfg)
14
15
```

22. assistant (2026-07-06T20:21:06.462Z)

Now let me confirm which output heatmap channel maps to the "current" frame by reading the online tracker and the runner that consumes `run_tensor` results, plus `read_image` and `utils/utils` (`_nms`, `_top1`).

23. user (2026-07-06T20:21:10.458Z)

```
1       import numpy as np
2
```

```
3       class Track:
4           def __init__(self):
5               self._xy_dict    = {}
6               self._score_dict = {}
7               self._visi_dict  = {}
8
9           def add(self, fid, x, y, visi, score):
10              self._xy_dict[fid]    = np.array([x,y])
11              self._visi_dict[fid]  = visi
12              self._score_dict[fid] = score
13
14          def is_visible(self, fid):
15              if not fid in self._visi_dict.keys():
16                  #raise KeyError('fid {} not found'.format(fid))
17                  return False
18              return self._visi_dict[fid]
19
20          @property
21          def last_fid(self):
22              fids = list( self._xy_dict.keys() )
23              return max(fids)
24
25          def xy(self, fid):
26              if not fid in self._xy_dict.keys():
27                  raise KeyError('fid {} not found'.format(fid))
28              return self._xy_dict[fid]
29
30          def predict(self, last_fid):
31              fid1 = last_fid
32              fid2 = fid1 - 1
33              fid3 = fid1 - 2
34              if self.is_visible(fid1) and self.is_visible(fid2) and self.is_visible(fid3):
35                  xy1 = self._xy_dict[fid1]
36                  xy2 = self._xy_dict[fid2]
37                  xy3 = self._xy_dict[fid3]
38                  acc     = (xy1-xy2) - (xy2-xy3)
39                  vel     = (xy1-xy2) + acc
40                  xy_pred = xy1 + vel + acc / 2
41              else:
42                  xy_pred = None
43              return xy_pred
44
45      class OnlineTracker:
46          def __init__(self, cfg):
47              self._max_disp = cfg['tracker']['max_disp']
48              self._fid      = 0
49              self._track    = Track()
50
51          def _select_best(self, frame_dets):
52              best_score = - np.Inf
53              visi       = False
54              x, y       = - np.Inf, - np.Inf
55
56              xy_pred = None
57
58              for det in frame_dets:
59                  score = det['score']
60                  if xy_pred is not None:
61                      qscore  = self._compute_quality(xy_pred, det['xy'],
self._track.xy(self._fid-1) )
62                      score  += qscore
63
64                  if score > best_score:
65                      best_score = score
66                      xy         = det['xy']
```

```
67                      x,y        = xy[0], xy[1]
68                      visi       = True
69              return x,y,visi,best_score
70
71          def _select_not_too_far(self, frame_dets):
72              if (self._fid==0) or (not self._track.is_visible(self._fid-1)):
73                  return frame_dets
74
75              frame_dets_ = []
76              for det in frame_dets:
77                  if np.linalg.norm( det['xy'] - self._track.xy(self._fid-1) ) <
self._max_disp:
78                      frame_dets_.append(det)
79              return frame_dets_
80
81          def _compute_quality(self, xy1, xy2, xy3):
82              return - np.linalg.norm( xy1-xy2 )
83
84          def update(self, frame_dets):
85              frame_dets     = self._select_not_too_far(frame_dets)
86              x,y,visi,score = self._select_best(frame_dets)
87              self._track.add(self._fid, x, y, visi, score)
88
89              self._fid += 1
90              return {'x': x, 'y': y, 'visi': visi, 'score': score}
91
92          def refresh(self):
93              self._fid   = 0
94              self._track = Track()
95
96
```

24. user (2026-07-06T20:21:13.485Z)

```
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/runners/eval.py:43:
frames_in  = detector.frames_in
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/runners/eval.py:55:
num_frames += imgs.shape[0] * frames_in
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/runners/eval.py:59:
batch_results, hms_vis = detector.run_tensor(imgs, trans)
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/runners/eval.py:117:
color_pred = (int(cm_pred(cnt2)[2]*255), int(cm_pred(cnt2)[1]*255), int(cm_pred(cnt2)[0]*255))
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/runners/eval.py:118:
color_gt   = (int(cm_gt(cnt2)[2]*255), int(cm_gt(cnt2)[1]*255), int(cm_gt(cnt2)[0]*255))
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/runners/eval.py:133:
color_pred = (int(cm_pred(cnt)[2]*255), int(cm_pred(cnt)[1]*255), int(cm_pred(cnt)[0]*255))
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/runners/eval.py:134:
color_gt   = (int(cm_gt(cnt)[2]*255), int(cm_gt(cnt)[1]*255), int(cm_gt(cnt)[0]*255))
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/utils/utils.py:22:def
read_image(img_path):
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/utils/utils.py:87:def
_nms(heat, kernel=3):
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/utils/utils.py:95:def
_top1(scores):
```

25. assistant (2026-07-06T20:21:16.840Z)

Let me read the eval runner around the `run_tensor` consumption to confirm how batch results are mapped to per-frame detections and which output channel corresponds to which frame, plus `read_image`.

26. user (2026-07-06T20:21:18.956Z)

```
40          if evaluator_all is not None:
41              evaluator  = Evaluator(cfg)
42
43          frames_in  = detector.frames_in
44          frames_out = detector.frames_out
45
46          # +---------------
47          t_start     = time.time()
48
49          det_results = defaultdict(list)
50          hm_results  = defaultdict(list)
51          rescale     = -1.
52          num_frames = 0
53          for batch_idx, (imgs, hms, trans, xys_gt, visis_gt, img_paths) in
enumerate(tqdm(dataloader, desc='[(CLIP-WISE INFERENCE)]' )):
54
55              num_frames += imgs.shape[0] * frames_in
56              if rescale < 0:
57                  rescale = trans[0][0,0,0].item()
58
59              batch_results, hms_vis = detector.run_tensor(imgs, trans)
60              img_paths   = [list(in_tuple) for in_tuple in img_paths]
61
62              for ib in batch_results.keys():
63                  for ie in batch_results[ib].keys():
64                      img_path    = img_paths[ie][ib]
65                      preds       = batch_results[ib][ie]
66                      det_results[img_path].extend(preds)
67                      hm_results[img_path].extend( hms_vis[ib][ie])
68
69          tracker.refresh()
70          result_dict = {}
71          for img_path, preds in det_results.items():
72              result_dict[img_path] = tracker.update(preds)
73
74          t_elapsed = time.time() - t_start
75          # +---------------
76          log.info('Time:{:.1f}(sec)'.format(t_elapsed))
77
78          cm_pred = plt.get_cmap('Reds', len(result_dict))
79          cm_gt   = plt.get_cmap('Greens', len(result_dict))
80
81          fp1_im_list = []
82          cnt = 0
83          for cnt, img_path in enumerate(result_dict.keys()):
84              xy_pred    = (result_dict[img_path]['x'], result_dict[img_path]['y'])
85              x_pred = result_dict[img_path]['x']
86              y_pred = result_dict[img_path]['y']
87              visi_pred  = result_dict[img_path]['visi']
88              score_pred = result_dict[img_path]['score']
89
90              center_gt = None
91              if gt is not None:
92                  center_gt = gt[img_path]
93
94              if (gt is not None) and (evaluator_all is not None):
95                  evaluator_all.eval_single_frame(xy_pred, visi_pred, score_pred,
```

```
center_gt.xy, center_gt.is_visible)
96                  result = evaluator.eval_single_frame(xy_pred, visi_pred, score_pred,
center_gt.xy, center_gt.is_visible)
97
98                  if result['fp1'] > 0 and result['se'] < rescale * dist_thresh:
99                      fp1_im_list.append(img_path)
100
101             if vis_frame_dir is not None:
102                 vis_frame_path = osp.join(vis_frame_dir, osp.basename(img_path)) if
vis_frame_dir is not None else None
103                 vis_gt         = cv2.imread(img_path)
104                 vis_pred       = cv2.imread(img_path)
105
106                 for cnt2, img_path2 in enumerate(result_dict.keys()):
107                     if cnt2 > cnt:
108                         break
109
110                     x_pred = result_dict[img_path2]['x']
111                     y_pred = result_dict[img_path2]['y']
112                     visi_pred  = result_dict[img_path2]['visi']
113                     score_pred = result_dict[img_path2]['score']
114
```

27. user (2026-07-06T20:21:20.218Z)

```
20          return np.sqrt(X2 - 2*XY + Y2)
21
22      def read_image(img_path):
23          """Keep reading image until succeed.
24          This can avoid IOError incurred by heavy IO process."""
25          got_img = False
26          if not osp.exists(img_path):
27              raise IOError("{} does not exist".format(img_path))
28          while not got_img:
29              try:
30                  img = Image.open(img_path).convert('RGB')
31                  got_img = True
32              except IOError:
33                  print("IOError incurred when reading '{}'. Will redo. Don't worry. Just
chill.".format(img_path))
34                  pass
35          return img
36
37      def list2txt(list_data):
38          txt = ''
39          for cnt, d in enumerate(list_data):
40              txt += '{}'.format(d)
41              if cnt<len(list_data)-1:
42                  txt += '-'
43          # print(txt)
44          return txt
45
46      def count_params(model, only_trainable=True):
47          if only_trainable:
48              num_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
49          else:
```

28. assistant (2026-07-06T20:21:26.270Z)

Now I understand the full picture. The critical insight on channel?frame mapping: the sampler builds sequences. Let me check the sampler to confirm the frame-to-channel correspondence ? specifically whether the 3 output heatmaps correspond to the 3 input frames in order (so channel `j` ? frame `j` in the window), and confirm how `img_paths[ie][ib]` maps `ie` (element index within the window) to a frame.

29. user (2026-07-06T20:21:32.794Z)

```
__init__.py
samplers.py
===
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:42:
clip_dir = osp.join(root_dir, match, frame_dirname, clip_name)
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:62:
self._frames_in  = cfg['model']['frames_in']
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:63:
self._frames_out = cfg['model']['frames_out']
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:64:
self._step       = cfg['detector']['step']
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:76:
train_outputs = self._gen_seq_list(self._train_matches, self._train_num_clip_ratio)
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:78:
self._train_num_frames         = train_outputs['num_frames']
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:79:
self._train_num_frames_with_gt = train_outputs['num_frames_with_gt']
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:85:
self._train_clips              = train_outputs['clip_seq_list_dict']
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:86:
self._train_clip_gts           = train_outputs['clip_seq_gt_dict_dict']
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:87:
self._train_clip_disps         = train_outputs['clip_seq_disps']
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:94:
test_outputs  = self._gen_seq_list(self._test_matches, self._test_num_clip_ratio)
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:96:
self._test_num_frames          = test_outputs['num_frames']
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:97:
self._test_num_frames_with_gt  = test_outputs['num_frames_with_gt']
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:103:
self._test_clips               = test_outputs['clip_seq_list_dict']
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:104:
self._test_clip_gts            = test_outputs['clip_seq_gt_dict_dict']
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:105:
self._test_clip_disps          = test_outputs['clip_seq_disps']
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:113:
log.info("train                  | {:7d} | {:7d} | {:13d} | {:7d} | {:7d} |
{:2.1f}+/-{:2.1f}".format(len(self._train_all), self._train_num_frames,
self._train_num_frames_with_gt, self._train_num_rallies, self._train_num_matches,
self._train_disp_mean, self._train_disp_std ) )
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:116:
num_frames_all         = 0
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:117:
num_frames_with_gt_all = 0
```

```
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:122:
num_frames = 0
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:124:
num_frames += len( tmp['frames'] )
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:125:
num_frames_with_gt = num_frames
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:128:
log.info("{} | {:7d} | {:7d} | {:13d} |         |         | {:2.1f}+/-{:2.1f}".format(clip_name,
num_items, num_frames, num_frames_with_gt, np.mean(disps), np.std(disps) ))
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:130:
num_frames_all         += num_frames
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:131:
num_frames_with_gt_all += num_frames_with_gt
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:134:
log.info("all                 | {:7d} | {:7d} | {:13d} | {:7d} |         |
{:2.1f}+/-{:2.1f}".format(num_items_all, num_frames_all, num_frames_with_gt_all, num_clips_all,
np.mean(disps_all), np.std(disps_all) ))
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:137:
log.info("test                    | {:7d} | {:7d} | {:13d} | {:7d} | {:7d} |
{:2.1f}+/-{:2.1f}".format(len(self._test_all), self._test_num_frames,
self._test_num_frames_with_gt, self._test_num_rallies, self._test_num_matches,
self._test_disp_mean, self._test_disp_std) )
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:141:
num_frames_all         = 0
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:142:
num_frames_with_gt_all = 0
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/WASB-SBDT/src/datasets/badminton.py:147:
num_frames = 0
```

30. assistant (2026-07-06T20:21:35.885Z)

Let me look at the seq_list generation in badminton.py to confirm the window ordering and channel-to-frame correspondence.

31. user (2026-07-06T20:21:38.734Z)

```
62:        self._frames_in  = cfg['model']['frames_in']
76:            train_outputs = self._gen_seq_list(self._train_matches,
self._train_num_clip_ratio)
78:            self._train_num_frames         = train_outputs['num_frames']
94:            test_outputs  = self._gen_seq_list(self._test_matches, self._test_num_clip_ratio)
96:            self._test_num_frames          = test_outputs['num_frames']
124:                    num_frames += len( tmp['frames'] )
149:                    num_frames += len( tmp['frames'] )
162:    def _gen_seq_list(self, matches, num_clip_ratio):
189:                for i in range(len(ball_xyvs)-self._frames_in+1):
190:                    names = frame_names[i:i+self._frames_in]
192:                    annos = [ ball_xyvs[j] for j in range(i+self._frames_in-
self._frames_out, i+self._frames_in)]
193:                    seq_list.append( {'frames': paths, 'annos': annos, 'match': match,
'clip': clip_name})
195:                        clip_seq_list.append( {'frames': paths, 'annos': annos, 'match':
match, 'clip': clip_name})
200:                for i in range(len(ball_xyvs)-1):
```

```
205:                         disps.append(disp)
206:                         clip_disps.append(disp)
208:                 for i in range(len(ball_xyvs)):
```

32. assistant (2026-07-06T20:21:43.538Z)

This confirms the exact windowing. Let me read lines 185-200 to nail the frame?channel mapping precisely.

33. user (2026-07-06T20:21:44.846Z)

```
184                     ball_xyvs = load_csv(clip_csv_path, frame_dir=clip_frame_dir)
185                     frame_names = os.listdir(clip_frame_dir)
186                     frame_names.sort()
187                     num_frames += len(frame_names)
188                     num_frames_with_gt += len(ball_xyvs)
189                     for i in range(len(ball_xyvs)-self._frames_in+1):
190                         names = frame_names[i:i+self._frames_in]
191                         paths = [ osp.join(clip_frame_dir, name) for name in names]
192                         annos = [ ball_xyvs[j] for j in range(i+self._frames_in-
self._frames_out, i+self._frames_in)]
193                         seq_list.append( {'frames': paths, 'annos': annos, 'match': match,
'clip': clip_name})
194                         if i%self._step==0:
195                             clip_seq_list.append( {'frames': paths, 'annos': annos, 'match':
match, 'clip': clip_name})
196                     clip_seq_list_dict[(match, clip_name)] = clip_seq_list
197
198                     # compute disp between consecutive frames
199                     clip_disps = []
```

34. assistant (2026-07-06T20:21:54.422Z)

This fully confirms the channel?frame mapping. With `frames_in=3`, `frames_out=3`:
- Window `i` = frames `[i, i+1, i+2]`, concatenated on channel dim ? 9-ch input.
- `annos` (targets) = `ball_xyvs[i : i+3]` ? so output channel `j` corresponds to input frame `i+j`.
- Therefore **output channel index 1 (middle) ? the middle frame** of the window. The prompt's assumption is correct.

Note the WASB inference is actually windowed at `step=3` (non-overlapping windows in the reference eval), but the prompt explicitly asks for **stride-1 sliding windows** and to take the middle channel per window. That's a valid per-frame detection scheme and matches the frame?channel semantics I confirmed.

I now have everything. Note one important subtlety: WASB preprocessing uses an **affine warp** (`get_affine_transform`: center=(w/2,h/2), scale=max(h,w)), which produces a centered, aspect-preserving letterbox ? NOT a plain stretch-resize to 512×288. For 1080p (1920×1080), scale=1920, so the 1920×1080 frame maps into a 512×288 box, and the letterbox happens to be full-width but the vertical extent maps 1080/1920*512 = 288 ? exactly fills height too (1920/1080 = 1.777 vs 512/288 = 1.777). So for 16:9 input it's effectively a plain resize. I'll note this and use plain resize for the fast paths, which is exact for 16:9 g1.mp4.

Let me write the script.

35. assistant (2026-07-06T20:23:40.555Z)

```
#!/usr/bin/env python3
"""
exp_t2.py -- torch-2.x GPU-resident-decode spike for the WASB shuttle detector.

Runs on a Linux L4 GPU VM. Assumes file layout under ~/t2:
    ~/t2/WASB-SBDT          (cloned repo; src/ on this path is importable)
    ~/t2/wasb.pth.tar       (weights: {"model_state_dict": <428-tensor state_dict>})
```

```
    ~/t2/g1.mp4             (golden clip, 1080p ~59.94 fps)
    ~/t2/g1_traj_baseline.csv  (cached baseline trajectory; columns Frame,Visibility,X,Y)

Answers:
  Q1 -- can we build WASB HRNet under torch 2.x and load the weights?
  Q2 -- throughput of decode->resize->normalize->HRNet-forward for three front-ends:
        (a) CPU (cv2)  (b) GPU torchcodec  (c) GPU PyNvVideoCodec
  Q3 -- quality parity of heatmaps + argmax shuttle position between CPU and a GPU variant.

Nothing here trains; everything is eval/inference only. Each variant is isolated in its own
try/except so one failure does not abort the others.
"""

import os
import sys
import time
import traceback

import numpy as np

# ---------------------------------------------------------------------------
# Config / paths
# ---------------------------------------------------------------------------

T2         = os.path.expanduser("~/t2")
REPO_DIR   = os.path.join(T2, "WASB-SBDT")
SRC_DIR    = os.path.join(REPO_DIR, "src")
WEIGHTS    = os.path.join(T2, "wasb.pth.tar")
VIDEO      = os.path.join(T2, "g1.mp4")
BASELINE   = os.path.join(T2, "g1_traj_baseline.csv")

N_FRAMES     = 1500      # subset of frames to process for throughput
N_PARITY     = 200       # first-N frames used for CPU-vs-GPU parity
BATCH_WINDOWS = 8        # 3-frame windows per forward pass
FRAMES_IN    = 3         # 3 RGB frames -> 9 input channels
IN_W, IN_H   = 512, 288  # model input (WxH)

# WASB preprocessing constants (confirmed from src/dataloaders/img_transforms.py:
#   build_img_transforms -> ToTensor() [/255, HWC->CHW, RGB] then
#   Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]))
IMAGENET_MEAN = np.array([0.485, 0.456, 0.406], dtype=np.float32)
IMAGENET_STD  = np.array([0.229, 0.224, 0.225], dtype=np.float32)

# Output heatmap channel used for the "current" frame's position.
# Confirmed from datasets/badminton.py: a window at index i covers frames [i, i+1, i+2]
# and its targets are ball_xyvs[i : i+3], so output channel j <-> input frame i+j.
# The middle frame of a 3-frame window is channel index 1.
CURRENT_CHANNEL = 1

# Make the repo importable (its modules import as top-level: models, dataloaders, utils...
sys.path.insert(0, SRC_DIR)

import torch  # noqa: E402
import torch.nn.functional as F  # noqa: E402


# ---------------------------------------------------------------------------
```

Model construction (uses WASB's own HRNet class; params hard-wired from wasb.ya

----

We instantiate HRNet directly rather than composing the Hydra config, to avoid Hydra runtime friction. The HRNet ctor reads cfg via both dict-style (cfg['...']) and attribute-style (cfg.MODEL.EXTRA...), so we pass an OmegaConf DictConfig, which supports both. Values are copied verbatim from src/configs/model/wasb.yaml.

```
WASB_MODEL_CFG = {
    "name": "hrnet",
    "frames_in": 3,
    "frames_out": 3,
    "inp_height": 288,
    "inp_width": 512,
    "out_height": 288,
    "out_width": 512,
    "rgb_diff": False,
    "out_scales": [0],
    "MODEL": {
        "EXTRA": {
            "FINAL_CONV_KERNEL": 1,
            "PRETRAINED_LAYERS": ["*"],
            "STEM": {"INPLANES": 64, "STRIDES": [1, 1]},
            "STAGE1": {"NUM_MODULES": 1, "NUM_BRANCHES": 1, "BLOCK": "BOTTLENECK",
                       "NUM_BLOCKS": [1], "NUM_CHANNELS": [32], "FUSE_METHOD": "SUM"},
            "STAGE2": {"NUM_MODULES": 1, "NUM_BRANCHES": 2, "BLOCK": "BASIC",
                       "NUM_BLOCKS": [2, 2], "NUM_CHANNELS": [16, 32], "FUSE_METHOD": "SUM"},
            "STAGE3": {"NUM_MODULES": 1, "NUM_BRANCHES": 3, "BLOCK": "BASIC",
                       "NUM_BLOCKS": [2, 2, 2], "NUM_CHANNELS": [16, 32, 64], "FUSE_METHOD":
"SUM"},
            "STAGE4": {"NUM_MODULES": 1, "NUM_BRANCHES": 4, "BLOCK": "BASIC",
                       "NUM_BLOCKS": [2, 2, 2, 2], "NUM_CHANNELS": [16, 32, 64, 128],
"FUSE_METHOD": "SUM"},
            "DECONV": {"NUM_DECONVS": 0, "KERNEL_SIZE": [], "NUM_BASIC_BLOCKS": 2},
        },
        "INIT_WEIGHTS": True,
    },
}


def build_wasb_hrnet():
    """Build the WASB HRNet from its own class and load the pretrained weights.

    Returns model on cuda in eval() mode.
    """
    from omegaconf import OmegaConf
    from models.hrnet import HRNet  # WASB's own HRNet (do NOT reimplement)

    cfg = OmegaConf.create(WASB_MODEL_CFG)
    model = HRNet(cfg)

    # torch >=2.6 defaults weights_only=True; a plain tensor state_dict loads fine,
    # but pass weights_only=False to be safe with the {"model_state_dict": ...} wrapper.
    try:
        ckpt = torch.load(WEIGHTS, map_location="cpu", weights_only=False)
    except TypeError:
        # older torch without weights_only kwarg
        ckpt = torch.load(WEIGHTS, map_location="cpu")

    state_dict = ckpt["model_state_dict"] if isinstance(ckpt, dict) and "model_state_dict" in
ckpt else ckpt
    missing, unexpected = model.load_state_dict(state_dict, strict=True)
    model.eval().cuda()
```

```
    return model


def hrnet_forward(model, x):
    """Run HRNet and return the scale-0 heatmap logits tensor (B, frames_out, H, W)."""
    out = model(x)          # dict {scale: (B, frames_out, H, W)} logits
    return out[0]
```

GPU normalize helper

```
_MEAN_T = torch.tensor(IMAGENET_MEAN, device="cuda").view(1, 3, 1, 1)
_STD_T  = torch.tensor(IMAGENET_STD,  device="cuda").view(1, 3, 1, 1)


def gpu_normalize_uint8_frames(frames_u8):
    """frames_u8: (K,3,H0,W0) uint8 RGB on cuda -> (K,3,IN_H,IN_W) float32 normalized on cuda.

    Mirrors ToTensor (/255) + Normalize(ImageNet). Resize is bilinear to match cv2.INTER_LINEAR.
    """
    x = frames_u8.float().div_(255.0)
    x = F.interpolate(x, size=(IN_H, IN_W), mode="bilinear", align_corners=False)
    x = (x - _MEAN_T) / _STD_T
    return x


def windows_from_frames(norm_frames):
    """norm_frames: list/tensor of per-frame (3,IN_H,IN_W) tensors on cuda.
    Build stride-1 3-frame windows concatenated on channel dim -> (num_windows, 9, IN_H, IN_W).
    """
    # norm_frames is a (M,3,H,W) tensor
    M = norm_frames.shape[0]
    if M < FRAMES_IN:
        return None
    wins = [norm_frames[i:i + FRAMES_IN].reshape(1, FRAMES_IN * 3, IN_H, IN_W)
            for i in range(M - FRAMES_IN + 1)]
    return torch.cat(wins, dim=0)
```

Heatmap / argmax utilities (parity)

```
def heatmap_current(logits):
    """logits: (B, frames_out, H, W) -> sigmoid heatmap of the current channel, (B,H,W) on cuda.
    Postprocessor applies sigmoid_() before decode, so we match that here.
    """
    return torch.sigmoid(logits[:, CURRENT_CHANNEL, :, :])


def argmax_xy(hm_2d):
    """hm_2d: (H,W) numpy -> (x,y) argmax in model (512x288) space."""
    idx = int(np.argmax(hm_2d))
    y, x = np.unravel_index(idx, hm_2d.shape)
    return float(x), float(y)
```

Variant (a): CPU front-end (cv2)

```
# ---------------------------------------------------------------------------
def run_cpu_frontend(model, n_frames, collect_parity=0):
    """Decode with cv2, resize+RGB+normalize on CPU, move to cuda, HRNet forward.

    Returns (fps, parity_dict) where parity_dict maps middle-frame-global-index ->
    (heatmap_2d_numpy, (x,y)) for the first `collect_parity` current-frames.
    """
    import cv2

    cap = cv2.VideoCapture(VIDEO)
    if not cap.isOpened():
        raise RuntimeError("cv2 could not open {}".format(VIDEO))

    parity = {}
    processed_windows = 0
    processed_current_frames = 0

    torch.cuda.synchronize()
    t0 = time.time()

    frame_buf = []          # rolling list of normalized (3,H,W) cpu tensors
    global_idx = 0          # index of the newest frame appended
    read_count = 0

    def flush(batch_windows_np, base_middle_indices):
        nonlocal processed_windows, processed_current_frames
        if not batch_windows_np:
            return
        x = torch.from_numpy(np.stack(batch_windows_np, axis=0)).cuda(non_blocking=True)
        with torch.no_grad():
            logits = hrnet_forward(model, x)
            hm = heatmap_current(logits)  # (B,H,W)
        processed_windows += x.shape[0]
        processed_current_frames += x.shape[0]
        if collect_parity:
            hm_np = hm.detach().cpu().numpy()
            for k, mid_idx in enumerate(base_middle_indices):
                if mid_idx < collect_parity and mid_idx not in parity:
                    parity[mid_idx] = (hm_np[k], argmax_xy(hm_np[k]))

    batch_windows_np = []
    batch_mid_indices = []

    while read_count < n_frames:
        ret, bgr = cap.read()
        if not ret:
            break
        rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
        rgb = cv2.resize(rgb, (IN_W, IN_H), interpolation=cv2.INTER_LINEAR)
        f = rgb.astype(np.float32) / 255.0
        f = (f - IMAGENET_MEAN) / IMAGENET_STD    # HWC
        f = np.transpose(f, (2, 0, 1))            # CHW
        frame_buf.append(f)
        global_idx = read_count
        read_count += 1

        if len(frame_buf) >= FRAMES_IN:
            # newest window covers frames [global_idx-2, global_idx-1, global_idx]
            win = np.concatenate(frame_buf[-FRAMES_IN:], axis=0)   # (9,H,W)
            mid_global = global_idx - 1                            # middle frame index
            batch_windows_np.append(win)
            batch_mid_indices.append(mid_global)
            if len(frame_buf) > FRAMES_IN:
                frame_buf.pop(0)
            if len(batch_windows_np) >= BATCH_WINDOWS:
```

```
                flush(batch_windows_np, batch_mid_indices)
                batch_windows_np, batch_mid_indices = [], []

    flush(batch_windows_np, batch_mid_indices)
    cap.release()

    torch.cuda.synchronize()
    dt = time.time() - t0
    fps = read_count / dt if dt > 0 else float("nan")
    return fps, parity, read_count
```

Variant (b): GPU-resident torchcodec

```
def run_torchcodec_frontend(model, n_frames, device="cuda", collect_parity=0):
    """Decode with torchcodec on `device`; resize+normalize on GPU; HRNet forward.

    Returns (fps, parity_dict, processed_frames, actual_device).
    """
    from torchcodec.decoders import VideoDecoder

    dec = VideoDecoder(VIDEO, device=device)
    total = dec.metadata.num_frames or n_frames
    n = min(n_frames, total)

    parity = {}
    torch.cuda.synchronize()
    t0 = time.time()

    frame_buf = []          # rolling list of normalized (3,H,W) cuda tensors
    batch_windows = []
    batch_mid = []
    processed = 0

    def flush():
        nonlocal processed
        if not batch_windows:
            return
        x = torch.stack(batch_windows, dim=0)           # (B,9,H,W) cuda
        with torch.no_grad():
            logits = hrnet_forward(model, x)
            hm = heatmap_current(logits)
        processed += x.shape[0]
        if collect_parity:
            hm_np = hm.detach().cpu().numpy()
            for k, mid_idx in enumerate(batch_mid):
                if mid_idx < collect_parity and mid_idx not in parity:
                    parity[mid_idx] = (hm_np[k], argmax_xy(hm_np[k]))

    for i in range(n):
        frame = dec[i]                    # (3,H0,W0) uint8 tensor on `device`
        if frame.device.type != "cuda":
            frame = frame.cuda(non_blocking=True)
        norm = gpu_normalize_uint8_frames(frame.unsqueeze(0))[0]   # (3,IN_H,IN_W) cuda
        frame_buf.append(norm)
        if len(frame_buf) >= FRAMES_IN:
            win = torch.cat(frame_buf[-FRAMES_IN:], dim=0)          # (9,H,W)
            batch_windows.append(win)
            batch_mid.append(i - 1)                                 # middle frame index
            if len(frame_buf) > FRAMES_IN:
                frame_buf.pop(0)
            if len(batch_windows) >= BATCH_WINDOWS:
```

```
                flush()
                batch_windows, batch_mid = [], []

    flush()
    torch.cuda.synchronize()
    dt = time.time() - t0
    fps = n / dt if dt > 0 else float("nan")
    return fps, parity, n, device
```

Variant (c): GPU-resident PyNvVideoCodec (DLPack -> cuda tensor)

```
def run_pynvvideocodec_frontend(model, n_frames, collect_parity=0):
    """Decode with PyNvVideoCodec to a cuda tensor via DLPack; GPU resize+normalize; forward.

    PyNvVideoCodec yields NV12 surfaces by default. We request RGB via the decoder's
    output-format if available; otherwise convert. API surface varies across versions,
    so this is wrapped defensively.
    """
    import PyNvVideoCodec as nvc

    parity = {}

    # Try the high-level SimpleDecoder first (newer API), fall back to ThreadedDecoder.
    dec = None
    api = None
    try:
        dec = nvc.SimpleDecoder(VIDEO, gpu_id=0, output_color_type=nvc.OutputColorType.RGB)
        api = "SimpleDecoder"
    except Exception:
        dec = nvc.SimpleDecoder(VIDEO)      # default color type; convert later
        api = "SimpleDecoder(default-color)"

    torch.cuda.synchronize()
    t0 = time.time()

    frame_buf = []
    batch_windows = []
    batch_mid = []
    processed = 0
    n = n_frames

    def to_cuda_rgb_uint8(surf):
        """surf: PyNvVideoCodec frame -> (3,H,W) uint8 cuda tensor (RGB)."""
        t = torch.from_dlpack(surf)                 # zero-copy from GPU surface
        # Expected either (H,W,3) or (3,H,W). Normalize to (3,H,W).
        if t.dim() == 3 and t.shape[-1] == 3:
            t = t.permute(2, 0, 1).contiguous()
        if t.dtype != torch.uint8:
            t = t.to(torch.uint8)
        return t

    def flush():
        nonlocal processed
        if not batch_windows:
            return
        x = torch.stack(batch_windows, dim=0)
        with torch.no_grad():
            logits = hrnet_forward(model, x)
            hm = heatmap_current(logits)
        processed += x.shape[0]
        if collect_parity:
```

```
            hm_np = hm.detach().cpu().numpy()
            for k, mid_idx in enumerate(batch_mid):
                if mid_idx < collect_parity and mid_idx not in parity:
                    parity[mid_idx] = (hm_np[k], argmax_xy(hm_np[k]))

    # SimpleDecoder is indexable / iterable depending on version; handle both.
    def frame_iter():
        try:
            length = len(dec)
        except Exception:
            length = None
        if length is not None:
            for i in range(min(n, length)):
                yield i, dec[i]
        else:
            i = 0
            for f in dec:
                if i >= n:
                    break
                yield i, f
                i += 1

    for i, surf in frame_iter():
        frame_u8 = to_cuda_rgb_uint8(surf)                      # (3,H0,W0) cuda uint8
        if frame_u8.device.type != "cuda":
            frame_u8 = frame_u8.cuda(non_blocking=True)
        norm = gpu_normalize_uint8_frames(frame_u8.unsqueeze(0))[0]
        frame_buf.append(norm)
        if len(frame_buf) >= FRAMES_IN:
            win = torch.cat(frame_buf[-FRAMES_IN:], dim=0)
            batch_windows.append(win)
            batch_mid.append(i - 1)
            if len(frame_buf) > FRAMES_IN:
                frame_buf.pop(0)
            if len(batch_windows) >= BATCH_WINDOWS:
                flush()
                batch_windows, batch_mid = [], []

    flush()
    torch.cuda.synchronize()
    dt = time.time() - t0
    processed_frames = min(n, i + 1) if 'i' in dir() else 0
    fps = processed_frames / dt if dt > 0 else float("nan")
    return fps, parity, processed_frames, api


# ---------------------------------------------------------------------------
# Q3 parity comparison
# ---------------------------------------------------------------------------

def compare_parity(parity_cpu, parity_gpu):
    """Compare heatmaps + argmax positions on shared middle-frame indices."""
    shared = sorted(set(parity_cpu) & set(parity_gpu))
    if not shared:
        return None
    hm_abs_means, hm_abs_maxes, px_dists = [], [], []
    for idx in shared:
        hm_c, xy_c = parity_cpu[idx]
        hm_g, xy_g = parity_gpu[idx]
        d = np.abs(hm_c.astype(np.float64) - hm_g.astype(np.float64))
        hm_abs_means.append(d.mean())
        hm_abs_maxes.append(d.max())
        px_dists.append(float(np.hypot(xy_c[0] - xy_g[0], xy_c[1] - xy_g[1])))
    return {
```

```
        "n": len(shared),
        "hm_mean_abs": float(np.mean(hm_abs_means)),
        "hm_max_abs": float(np.max(hm_abs_maxes)),
        "argmax_median_px": float(np.median(px_dists)),
        "argmax_max_px": float(np.max(px_dists)),
    }
```

Main

```
def main():
    print("=" * 78, flush=True)
    print("exp_t2.py -- WASB GPU-resident-decode spike", flush=True)
    print("=" * 78, flush=True)
    print("torch: {}   cuda avail: {}   device: {}".format(
        torch.__version__, torch.cuda.is_available(),
        torch.cuda.get_device_name(0) if torch.cuda.is_available() else "N/A"), flush=True)
    print("N_FRAMES={}  N_PARITY={}  BATCH_WINDOWS={}  input {}x{} (WxH)
current_channel={}".format(
        N_FRAMES, N_PARITY, BATCH_WINDOWS, IN_W, IN_H, CURRENT_CHANNEL), flush=True)

    results = {
        "q1_pass": False,
        "fps_a": None, "fps_b": None, "fps_c": None,
        "b_device": None, "c_api": None,
        "q3": None, "q3_gpu_variant": None,
    }

    # ---- Q1 ------------------------------------------------------------
    print("\n" + "-" * 78, flush=True)
    print("[Q1] Build WASB HRNet under torch 2.x + load weights", flush=True)
    print("-" * 78, flush=True)
    model = None
    try:
        model = build_wasb_hrnet()
        dummy = torch.zeros(1, 9, IN_H, IN_W, device="cuda")
        with torch.no_grad():
            out = hrnet_forward(model, dummy)
        print("torch version: {}".format(torch.__version__), flush=True)
        print("WASB HRNet built + weights loaded OK", flush=True)
        print("dummy forward output shape (scale 0): {}".format(tuple(out.shape)), flush=True)
        results["q1_pass"] = True
    except Exception:
        print("[Q1] FAILED:", flush=True)
        traceback.print_exc()
        print("\nQ1 failed -> cannot run Q2/Q3. Exiting.", flush=True)
        _print_summary(results)
        return

    # ---- Q2a: CPU ------------------------------------------------------
    print("\n" + "-" * 78, flush=True)
    print("[Q2a] CPU front-end (cv2 decode -> cv2 resize -> RGB -> CPU normalize -> cuda ->
HRNet)", flush=True)
    print("-" * 78, flush=True)
    parity_cpu = {}
    try:
        fps_a, parity_cpu, n_read = run_cpu_frontend(model, N_FRAMES, collect_parity=N_PARITY)
        results["fps_a"] = fps_a
        print("[Q2a] processed {} frames  fps = {:.2f}".format(n_read, fps_a), flush=True)
    except Exception:
        print("[Q2a] FAILED:", flush=True)
        traceback.print_exc()
```

```
    # ---- Q2b: torchcodec ------------------------------------------------
    print("\n" + "-" * 78, flush=True)
    print("[Q2b] GPU-resident torchcodec (device=cuda) -> GPU resize/normalize -> HRNet",
flush=True)
    print("-" * 78, flush=True)
    parity_b = {}
    b_worked = False
    try:
        fps_b, parity_b, n_b, dev = run_torchcodec_frontend(
            model, N_FRAMES, device="cuda", collect_parity=N_PARITY)
        results["fps_b"] = fps_b
        results["b_device"] = dev
        b_worked = True
        print("[Q2b] processed {} frames  device={}  fps = {:.2f}".format(n_b, dev, fps_b),
flush=True)
    except Exception:
        print("[Q2b] torchcodec device=cuda FAILED:", flush=True)
        traceback.print_exc()
        print("[Q2b] retrying torchcodec device=cpu (decode on CPU, resize/normalize on
GPU)...", flush=True)
        try:
            fps_b, parity_b, n_b, dev = run_torchcodec_frontend(
                model, N_FRAMES, device="cpu", collect_parity=N_PARITY)
            results["fps_b"] = fps_b
            results["b_device"] = dev
            b_worked = True
            print("[Q2b] (cpu-decode fallback) processed {} frames  fps = {:.2f}".format(n_b,
fps_b), flush=True)
        except Exception:
            print("[Q2b] torchcodec cpu fallback ALSO FAILED:", flush=True)
            traceback.print_exc()

    # ---- Q2c: PyNvVideoCodec --------------------------------------------
    print("\n" + "-" * 78, flush=True)
    print("[Q2c] GPU-resident PyNvVideoCodec (DLPack -> cuda) -> GPU resize/normalize -> HRNet",
flush=True)
    print("-" * 78, flush=True)
    parity_c = {}
    c_worked = False
    try:
        fps_c, parity_c, n_c, api = run_pynvvideocodec_frontend(model, N_FRAMES,
collect_parity=N_PARITY)
        results["fps_c"] = fps_c
        results["c_api"] = api
        c_worked = True
        print("[Q2c] processed {} frames  api={}  fps = {:.2f}".format(n_c, api, fps_c),
flush=True)
    except Exception:
        print("[Q2c] PyNvVideoCodec FAILED:", flush=True)
        traceback.print_exc()

    # ---- Q3: parity CPU vs whichever GPU variant worked -----------------
    print("\n" + "-" * 78, flush=True)
    print("[Q3] Quality parity: CPU vs GPU (heatmap deltas + argmax pixel distance)",
flush=True)
    print("-" * 78, flush=True)
    gpu_parity, gpu_name = None, None
    if b_worked and parity_b:
        gpu_parity, gpu_name = parity_b, "torchcodec(b)"
    elif c_worked and parity_c:
        gpu_parity, gpu_name = parity_c, "PyNvVideoCodec(c)"

    if parity_cpu and gpu_parity is not None:
        q3 = compare_parity(parity_cpu, gpu_parity)
        results["q3"] = q3
```

```
        results["q3_gpu_variant"] = gpu_name
        if q3 is None:
            print("[Q3] No overlapping frames to compare.", flush=True)
        else:
            print("[Q3] CPU vs {} over {} current-frames:".format(gpu_name, q3["n"]),
flush=True)
            print("      heatmap mean|delta| = {:.6e}".format(q3["hm_mean_abs"]), flush=True)
            print("      heatmap  max|delta| = {:.6e}".format(q3["hm_max_abs"]), flush=True)
            print("      argmax median px dist = {:.3f}".format(q3["argmax_median_px"]),
flush=True)
            print("      argmax    max px dist = {:.3f}".format(q3["argmax_max_px"]),
flush=True)
    else:
        print("[Q3] Skipped -- need CPU parity data and at least one working GPU variant.",
flush=True)

    _print_summary(results)


def _print_summary(r):
    print("\n" + "=" * 78, flush=True)
    print("SUMMARY", flush=True)
    print("=" * 78, flush=True)
    print("Q1 (build + load weights): {}".format("PASS" if r["q1_pass"] else "FAIL"),
flush=True)
    print("Q2 fps table:", flush=True)

    def fmt(v):
        return "{:.2f}".format(v) if isinstance(v, float) else "n/a (failed)"

    print("   (a) CPU cv2            : {}".format(fmt(r["fps_a"])), flush=True)
    print("   (b) torchcodec [{}]{}: {}".format(
        r["b_device"] or "?", " " * max(0, 6 - len(str(r["b_device"] or "?"))),
fmt(r["fps_b"])), flush=True)
    print("   (c) PyNvVideoCodec     : {}".format(fmt(r["fps_c"])), flush=True)
    if r["q3"]:
        print("Q3 (CPU vs {}):".format(r["q3_gpu_variant"]), flush=True)
        print("   heatmap mean|delta|   : {:.6e}".format(r["q3"]["hm_mean_abs"]), flush=True)
        print("   argmax median px dist : {:.3f}".format(r["q3"]["argmax_median_px"]),
flush=True)
    else:
        print("Q3: n/a", flush=True)
    print("=" * 78, flush=True)


if __name__ == "__main__":
    main()
```

36. user (2026-07-06T20:23:40.611Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/exp_t2.py (file state is current in your context ? no need to Read it back)

37. assistant (2026-07-06T20:23:46.279Z)

Let me verify the script parses cleanly (syntax only ? no torch/cuda here).

38. user (2026-07-06T20:23:48.572Z)

SYNTAX OK

39. assistant (2026-07-06T20:23:55.827Z)

Syntax is clean. One small correctness fix: in the PyNvVideoCodec variant the `'i' in dir()` guard is fragile ? `dir()` returns names in the local scope and `i` may not be defined if the iterator was empty. Let me make it robust.

40. user (2026-07-06T20:23:58.485Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/exp_t2.py has been updated successfully. (file state is current in your context ? no need to Read it back)

41. user (2026-07-06T20:24:05.941Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/exp_t2.py has been updated successfully. (file state is current in your context ? no need to Read it back)

42. user (2026-07-06T20:24:14.018Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/exp_t2.py has been updated successfully. (file state is current in your context ? no need to Read it back)

43. assistant (2026-07-06T20:24:18.304Z)

Also, `model.load_state_dict(..., strict=True)` returns `None` when strict succeeds (it doesn't return the `(missing, unexpected)` tuple ? that's only with `strict=False`). Let me fix that unpacking bug.

44. user (2026-07-06T20:24:21.759Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/dbc4cdae-04f6-4d06-af90-397bbce3fc57/scratchpad/exp_t2.py has been updated successfully. (file state is current in your context ? no need to Read it back)

45. user (2026-07-06T20:24:28.296Z)

SYNTAX OK

46. assistant (2026-07-06T20:26:32.692Z)

All edits are in place and the script parses. Here is the deliverable.

Note: I read WASB's own code to confirm every load-bearing detail ? the HRNet class/ctor (`src/models/hrnet.py`), how the detector loads weights (`torch.load(...)["model_state_dict"]` ? `load_state_dict`, `src/detectors/detector.py`), the exact preprocessing (`src/dataloaders/img_transforms.py` ? `ToTensor` /255 + ImageNet `Normalize`; RGB via PIL `.convert('RGB')` in `src/utils/utils.py`), the sigmoid-then-decode postprocessor (`src/detectors/postprocessor.py`), and the frame?channel mapping (`src/datasets/badminton.py` lines 189?195). The architecture params are copied verbatim from `src/configs/model/wasb.yaml`.

```python
#!/usr/bin/env python3
"""
exp_t2.py -- torch-2.x GPU-resident-decode spike for the WASB shuttle detector.

Runs on a Linux L4 GPU VM. Assumes file layout under ~/t2:
    ~/t2/WASB-SBDT          (cloned repo; src/ on this path is importable)
    ~/t2/wasb.pth.tar       (weights: {"model_state_dict": <428-tensor state_dict>})
    ~/t2/g1.mp4             (golden clip, 1080p ~59.94 fps)
    ~/t2/g1_traj_baseline.csv  (cached baseline trajectory; columns Frame,Visibility,X,Y)

Answers:
  Q1 -- can we build WASB HRNet under torch 2.x and load the weights?
  Q2 -- throughput of decode->resize->normalize->HRNet-forward for three front-ends:
```

```
        (a) CPU (cv2)  (b) GPU torchcodec  (c) GPU PyNvVideoCodec
  Q3 -- quality parity of heatmaps + argmax shuttle position between CPU and a GPU variant.

Nothing here trains; everything is eval/inference only. Each variant is isolated in its own
try/except so one failure does not abort the others.
"""

import os
import sys
import time
import traceback

import numpy as np

# ---------------------------------------------------------------------------
# Config / paths
# ---------------------------------------------------------------------------
T2         = os.path.expanduser("~/t2")
REPO_DIR   = os.path.join(T2, "WASB-SBDT")
SRC_DIR    = os.path.join(REPO_DIR, "src")
WEIGHTS    = os.path.join(T2, "wasb.pth.tar")
VIDEO      = os.path.join(T2, "g1.mp4")
BASELINE   = os.path.join(T2, "g1_traj_baseline.csv")

N_FRAMES     = 1500      # subset of frames to process for throughput
N_PARITY     = 200       # first-N frames used for CPU-vs-GPU parity
BATCH_WINDOWS = 8        # 3-frame windows per forward pass
FRAMES_IN    = 3         # 3 RGB frames -> 9 input channels
IN_W, IN_H   = 512, 288  # model input (WxH)

# WASB preprocessing constants (confirmed from src/dataloaders/img_transforms.py:
#   build_img_transforms -> ToTensor() [/255, HWC->CHW, RGB] then
#   Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]))
IMAGENET_MEAN = np.array([0.485, 0.456, 0.406], dtype=np.float32)
IMAGENET_STD  = np.array([0.229, 0.224, 0.225], dtype=np.float32)

# Output heatmap channel used for the "current" frame's position.
# Confirmed from datasets/badminton.py: a window at index i covers frames [i, i+1, i+2]
# and its targets are ball_xyvs[i : i+3], so output channel j <-> input frame i+j.
# The middle frame of a 3-frame window is channel index 1.
CURRENT_CHANNEL = 1

# Make the repo importable (its modules import as top-level: models, dataloaders, utils..
sys.path.insert(0, SRC_DIR)

import torch  # noqa: E402
import torch.nn.functional as F  # noqa: E402


# ---------------------------------------------------------------------------
# Model construction (uses WASB's own HRNet class; params hard-wired from wasb.ya
# ---------------------------------------------------------------------------
# We instantiate HRNet directly rather than composing the Hydra config, to avoid
# Hydra runtime friction. The HRNet ctor reads cfg via both dict-style (cfg['...'])
```

and attribute-style (cfg.MODEL.EXTRA...), so we pass an OmegaConf DictConfig, which supports both. Values are copied verbatim from src/configs/model/wasb.yaml.

```
WASB_MODEL_CFG = {
    "name": "hrnet",
    "frames_in": 3,
    "frames_out": 3,
    "inp_height": 288,
    "inp_width": 512,
    "out_height": 288,
    "out_width": 512,
    "rgb_diff": False,
    "out_scales": [0],
    "MODEL": {
        "EXTRA": {
            "FINAL_CONV_KERNEL": 1,
            "PRETRAINED_LAYERS": ["*"],
            "STEM": {"INPLANES": 64, "STRIDES": [1, 1]},
            "STAGE1": {"NUM_MODULES": 1, "NUM_BRANCHES": 1, "BLOCK": "BOTTLENECK",
                       "NUM_BLOCKS": [1], "NUM_CHANNELS": [32], "FUSE_METHOD": "SUM"},
            "STAGE2": {"NUM_MODULES": 1, "NUM_BRANCHES": 2, "BLOCK": "BASIC",
                       "NUM_BLOCKS": [2, 2], "NUM_CHANNELS": [16, 32], "FUSE_METHOD": "SUM"},
            "STAGE3": {"NUM_MODULES": 1, "NUM_BRANCHES": 3, "BLOCK": "BASIC",
                       "NUM_BLOCKS": [2, 2, 2], "NUM_CHANNELS": [16, 32, 64], "FUSE_METHOD":
"SUM"},
            "STAGE4": {"NUM_MODULES": 1, "NUM_BRANCHES": 4, "BLOCK": "BASIC",
                       "NUM_BLOCKS": [2, 2, 2, 2], "NUM_CHANNELS": [16, 32, 64, 128],
"FUSE_METHOD": "SUM"},
            "DECONV": {"NUM_DECONVS": 0, "KERNEL_SIZE": [], "NUM_BASIC_BLOCKS": 2},
        },
        "INIT_WEIGHTS": True,
    },
}


def build_wasb_hrnet():
    """Build the WASB HRNet from its own class and load the pretrained weights.

    Returns model on cuda in eval() mode.
    """
    from omegaconf import OmegaConf
    from models.hrnet import HRNet  # WASB's own HRNet (do NOT reimplement)

    cfg = OmegaConf.create(WASB_MODEL_CFG)
    model = HRNet(cfg)

    # torch >=2.6 defaults weights_only=True; a plain tensor state_dict loads fine,
    # but pass weights_only=False to be safe with the {"model_state_dict": ...} wrapper.
    try:
        ckpt = torch.load(WEIGHTS, map_location="cpu", weights_only=False)
    except TypeError:
        # older torch without weights_only kwarg
        ckpt = torch.load(WEIGHTS, map_location="cpu")

    state_dict = ckpt["model_state_dict"] if isinstance(ckpt, dict) and "model_state_dict" in
ckpt else ckpt
    print("state_dict tensors: {}".format(len(state_dict)), flush=True)
    model.load_state_dict(state_dict, strict=True)   # raises if any key mismatch
    model.eval().cuda()
    return model


def hrnet_forward(model, x):
    """Run HRNet and return the scale-0 heatmap logits tensor (B, frames_out, H, W)."""
    out = model(x)          # dict {scale: (B, frames_out, H, W)} logits
```

```
    return out[0]
```

GPU normalize helper

```
_MEAN_T = torch.tensor(IMAGENET_MEAN, device="cuda").view(1, 3, 1, 1)
_STD_T  = torch.tensor(IMAGENET_STD,  device="cuda").view(1, 3, 1, 1)


def gpu_normalize_uint8_frames(frames_u8):
    """frames_u8: (K,3,H0,W0) uint8 RGB on cuda -> (K,3,IN_H,IN_W) float32 normalized on cuda.

    Mirrors ToTensor (/255) + Normalize(ImageNet). Resize is bilinear to match cv2.INTER_LINEAR.
    """
    x = frames_u8.float().div_(255.0)
    x = F.interpolate(x, size=(IN_H, IN_W), mode="bilinear", align_corners=False)
    x = (x - _MEAN_T) / _STD_T
    return x


def windows_from_frames(norm_frames):
    """norm_frames: list/tensor of per-frame (3,IN_H,IN_W) tensors on cuda.
    Build stride-1 3-frame windows concatenated on channel dim -> (num_windows, 9, IN_H, IN_W).
    """
    # norm_frames is a (M,3,H,W) tensor
    M = norm_frames.shape[0]
    if M < FRAMES_IN:
        return None
    wins = [norm_frames[i:i + FRAMES_IN].reshape(1, FRAMES_IN * 3, IN_H, IN_W)
            for i in range(M - FRAMES_IN + 1)]
    return torch.cat(wins, dim=0)
```

Heatmap / argmax utilities (parity)

```
def heatmap_current(logits):
    """logits: (B, frames_out, H, W) -> sigmoid heatmap of the current channel, (B,H,W) on cuda.
    Postprocessor applies sigmoid_() before decode, so we match that here.
    """
    return torch.sigmoid(logits[:, CURRENT_CHANNEL, :, :])


def argmax_xy(hm_2d):
    """hm_2d: (H,W) numpy -> (x,y) argmax in model (512x288) space."""
    idx = int(np.argmax(hm_2d))
    y, x = np.unravel_index(idx, hm_2d.shape)
    return float(x), float(y)
```

Variant (a): CPU front-end (cv2)

```
def run_cpu_frontend(model, n_frames, collect_parity=0):
    """Decode with cv2, resize+RGB+normalize on CPU, move to cuda, HRNet forward.

    Returns (fps, parity_dict) where parity_dict maps middle-frame-global-index ->
```

```
(heatmap_2d_numpy, (x,y)) for the first `collect_parity` current-frames.
"""
import cv2

cap = cv2.VideoCapture(VIDEO)
if not cap.isOpened():
    raise RuntimeError("cv2 could not open {}".format(VIDEO))

parity = {}
processed_windows = 0
processed_current_frames = 0

torch.cuda.synchronize()
t0 = time.time()

frame_buf = []          # rolling list of normalized (3,H,W) cpu tensors
global_idx = 0          # index of the newest frame appended
read_count = 0

def flush(batch_windows_np, base_middle_indices):
    nonlocal processed_windows, processed_current_frames
    if not batch_windows_np:
        return
    x = torch.from_numpy(np.stack(batch_windows_np, axis=0)).cuda(non_blocking=True)
    with torch.no_grad():
        logits = hrnet_forward(model, x)
        hm = heatmap_current(logits)  # (B,H,W)
    processed_windows += x.shape[0]
    processed_current_frames += x.shape[0]
    if collect_parity:
        hm_np = hm.detach().cpu().numpy()
        for k, mid_idx in enumerate(base_middle_indices):
            if mid_idx < collect_parity and mid_idx not in parity:
                parity[mid_idx] = (hm_np[k], argmax_xy(hm_np[k]))

batch_windows_np = []
batch_mid_indices = []

while read_count < n_frames:
    ret, bgr = cap.read()
    if not ret:
        break
    rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
    rgb = cv2.resize(rgb, (IN_W, IN_H), interpolation=cv2.INTER_LINEAR)
    f = rgb.astype(np.float32) / 255.0
    f = (f - IMAGENET_MEAN) / IMAGENET_STD    # HWC
    f = np.transpose(f, (2, 0, 1))            # CHW
    frame_buf.append(f)
    global_idx = read_count
    read_count += 1

    if len(frame_buf) >= FRAMES_IN:
        # newest window covers frames [global_idx-2, global_idx-1, global_idx]
        win = np.concatenate(frame_buf[-FRAMES_IN:], axis=0)   # (9,H,W)
        mid_global = global_idx - 1                            # middle frame index
        batch_windows_np.append(win)
        batch_mid_indices.append(mid_global)
        if len(frame_buf) > FRAMES_IN:
            frame_buf.pop(0)
        if len(batch_windows_np) >= BATCH_WINDOWS:
            flush(batch_windows_np, batch_mid_indices)
            batch_windows_np, batch_mid_indices = [], []

flush(batch_windows_np, batch_mid_indices)
cap.release()
```

```
    torch.cuda.synchronize()
    dt = time.time() - t0
    fps = read_count / dt if dt > 0 else float("nan")
    return fps, parity, read_count
```

Variant (b): GPU-resident torchcodec

```
def run_torchcodec_frontend(model, n_frames, device="cuda", collect_parity=0):
    """Decode with torchcodec on `device`; resize+normalize on GPU; HRNet forward.

    Returns (fps, parity_dict, processed_frames, actual_device).
    """
    from torchcodec.decoders import VideoDecoder

    dec = VideoDecoder(VIDEO, device=device)
    total = dec.metadata.num_frames or n_frames
    n = min(n_frames, total)

    parity = {}
    torch.cuda.synchronize()
    t0 = time.time()

    frame_buf = []          # rolling list of normalized (3,H,W) cuda tensors
    batch_windows = []
    batch_mid = []
    processed = 0

    def flush():
        nonlocal processed
        if not batch_windows:
            return
        x = torch.stack(batch_windows, dim=0)           # (B,9,H,W) cuda
        with torch.no_grad():
            logits = hrnet_forward(model, x)
            hm = heatmap_current(logits)
        processed += x.shape[0]
        if collect_parity:
            hm_np = hm.detach().cpu().numpy()
            for k, mid_idx in enumerate(batch_mid):
                if mid_idx < collect_parity and mid_idx not in parity:
                    parity[mid_idx] = (hm_np[k], argmax_xy(hm_np[k]))

    for i in range(n):
        frame = dec[i]                    # (3,H0,W0) uint8 tensor on `device`
        if frame.device.type != "cuda":
            frame = frame.cuda(non_blocking=True)
        norm = gpu_normalize_uint8_frames(frame.unsqueeze(0))[0]   # (3,IN_H,IN_W) cuda
        frame_buf.append(norm)
        if len(frame_buf) >= FRAMES_IN:
            win = torch.cat(frame_buf[-FRAMES_IN:], dim=0)          # (9,H,W)
            batch_windows.append(win)
            batch_mid.append(i - 1)                                 # middle frame index
            if len(frame_buf) > FRAMES_IN:
                frame_buf.pop(0)
            if len(batch_windows) >= BATCH_WINDOWS:
                flush()
                batch_windows, batch_mid = [], []

    flush()
    torch.cuda.synchronize()
    dt = time.time() - t0
```

```
    fps = n / dt if dt > 0 else float("nan")
    return fps, parity, n, device
```

Variant (c): GPU-resident PyNvVideoCodec (DLPack -> cuda tensor)

```
def run_pynvvideocodec_frontend(model, n_frames, collect_parity=0):
    """Decode with PyNvVideoCodec to a cuda tensor via DLPack; GPU resize+normalize; forward.

    PyNvVideoCodec yields NV12 surfaces by default. We request RGB via the decoder's
    output-format if available; otherwise convert. API surface varies across versions,
    so this is wrapped defensively.
    """
    import PyNvVideoCodec as nvc

    parity = {}

    # Try the high-level SimpleDecoder first (newer API), fall back to ThreadedDecoder.
    dec = None
    api = None
    try:
        dec = nvc.SimpleDecoder(VIDEO, gpu_id=0, output_color_type=nvc.OutputColorType.RGB)
        api = "SimpleDecoder"
    except Exception:
        dec = nvc.SimpleDecoder(VIDEO)      # default color type; convert later
        api = "SimpleDecoder(default-color)"

    torch.cuda.synchronize()
    t0 = time.time()

    frame_buf = []
    batch_windows = []
    batch_mid = []
    processed = 0
    n = n_frames
    last_i = -1

    def to_cuda_rgb_uint8(surf):
        """surf: PyNvVideoCodec frame -> (3,H,W) uint8 cuda tensor (RGB)."""
        t = torch.from_dlpack(surf)                # zero-copy from GPU surface
        # Expected either (H,W,3) or (3,H,W). Normalize to (3,H,W).
        if t.dim() == 3 and t.shape[-1] == 3:
            t = t.permute(2, 0, 1).contiguous()
        if t.dtype != torch.uint8:
            t = t.to(torch.uint8)
        return t

    def flush():
        nonlocal processed
        if not batch_windows:
            return
        x = torch.stack(batch_windows, dim=0)
        with torch.no_grad():
            logits = hrnet_forward(model, x)
            hm = heatmap_current(logits)
        processed += x.shape[0]
        if collect_parity:
            hm_np = hm.detach().cpu().numpy()
            for k, mid_idx in enumerate(batch_mid):
                if mid_idx < collect_parity and mid_idx not in parity:
                    parity[mid_idx] = (hm_np[k], argmax_xy(hm_np[k]))
```

```
    # SimpleDecoder is indexable / iterable depending on version; handle both.
    def frame_iter():
        try:
            length = len(dec)
        except Exception:
            length = None
        if length is not None:
            for i in range(min(n, length)):
                yield i, dec[i]
        else:
            i = 0
            for f in dec:
                if i >= n:
                    break
                yield i, f
                i += 1

    for i, surf in frame_iter():
        last_i = i
        frame_u8 = to_cuda_rgb_uint8(surf)                      # (3,H0,W0) cuda uint8
        if frame_u8.device.type != "cuda":
            frame_u8 = frame_u8.cuda(non_blocking=True)
        norm = gpu_normalize_uint8_frames(frame_u8.unsqueeze(0))[0]
        frame_buf.append(norm)
        if len(frame_buf) >= FRAMES_IN:
            win = torch.cat(frame_buf[-FRAMES_IN:], dim=0)
            batch_windows.append(win)
            batch_mid.append(i - 1)
            if len(frame_buf) > FRAMES_IN:
                frame_buf.pop(0)
            if len(batch_windows) >= BATCH_WINDOWS:
                flush()
                batch_windows, batch_mid = [], []

    flush()
    torch.cuda.synchronize()
    dt = time.time() - t0
    processed_frames = last_i + 1
    fps = processed_frames / dt if dt > 0 else float("nan")
    return fps, parity, processed_frames, api


# ---------------------------------------------------------------------------
# Q3 parity comparison
# ---------------------------------------------------------------------------

def compare_parity(parity_cpu, parity_gpu):
    """Compare heatmaps + argmax positions on shared middle-frame indices."""
    shared = sorted(set(parity_cpu) & set(parity_gpu))
    if not shared:
        return None
    hm_abs_means, hm_abs_maxes, px_dists = [], [], []
    for idx in shared:
        hm_c, xy_c = parity_cpu[idx]
        hm_g, xy_g = parity_gpu[idx]
        d = np.abs(hm_c.astype(np.float64) - hm_g.astype(np.float64))
        hm_abs_means.append(d.mean())
        hm_abs_maxes.append(d.max())
        px_dists.append(float(np.hypot(xy_c[0] - xy_g[0], xy_c[1] - xy_g[1])))
    return {
        "n": len(shared),
        "hm_mean_abs": float(np.mean(hm_abs_means)),
        "hm_max_abs": float(np.max(hm_abs_maxes)),
        "argmax_median_px": float(np.median(px_dists)),
```

```
        "argmax_max_px": float(np.max(px_dists)),
    }
```

Main

```
def main():
    print("=" * 78, flush=True)
    print("exp_t2.py -- WASB GPU-resident-decode spike", flush=True)
    print("=" * 78, flush=True)
    print("torch: {}   cuda avail: {}   device: {}".format(
        torch.__version__, torch.cuda.is_available(),
        torch.cuda.get_device_name(0) if torch.cuda.is_available() else "N/A"), flush=True)
    print("N_FRAMES={}  N_PARITY={}  BATCH_WINDOWS={}  input {}x{} (WxH)
current_channel={}".format(
        N_FRAMES, N_PARITY, BATCH_WINDOWS, IN_W, IN_H, CURRENT_CHANNEL), flush=True)

    results = {
        "q1_pass": False,
        "fps_a": None, "fps_b": None, "fps_c": None,
        "b_device": None, "c_api": None,
        "q3": None, "q3_gpu_variant": None,
    }

    # ---- Q1 ------------------------------------------------------------
    print("\n" + "-" * 78, flush=True)
    print("[Q1] Build WASB HRNet under torch 2.x + load weights", flush=True)
    print("-" * 78, flush=True)
    model = None
    try:
        model = build_wasb_hrnet()
        dummy = torch.zeros(1, 9, IN_H, IN_W, device="cuda")
        with torch.no_grad():
            out = hrnet_forward(model, dummy)
        print("torch version: {}".format(torch.__version__), flush=True)
        print("WASB HRNet built + weights loaded OK", flush=True)
        print("dummy forward output shape (scale 0): {}".format(tuple(out.shape)), flush=True)
        results["q1_pass"] = True
    except Exception:
        print("[Q1] FAILED:", flush=True)
        traceback.print_exc()
        print("\nQ1 failed -> cannot run Q2/Q3. Exiting.", flush=True)
        _print_summary(results)
        return

    # ---- Q2a: CPU ------------------------------------------------------
    print("\n" + "-" * 78, flush=True)
    print("[Q2a] CPU front-end (cv2 decode -> cv2 resize -> RGB -> CPU normalize -> cuda ->
HRNet)", flush=True)
    print("-" * 78, flush=True)
    parity_cpu = {}
    try:
        fps_a, parity_cpu, n_read = run_cpu_frontend(model, N_FRAMES, collect_parity=N_PARITY)
        results["fps_a"] = fps_a
        print("[Q2a] processed {} frames  fps = {:.2f}".format(n_read, fps_a), flush=True)
    except Exception:
        print("[Q2a] FAILED:", flush=True)
        traceback.print_exc()

    # ---- Q2b: torchcodec -----------------------------------------------
    print("\n" + "-" * 78, flush=True)
    print("[Q2b] GPU-resident torchcodec (device=cuda) -> GPU resize/normalize -> HRNet",
```

```
flush=True)
    print("-" * 78, flush=True)
    parity_b = {}
    b_worked = False
    try:
        fps_b, parity_b, n_b, dev = run_torchcodec_frontend(
            model, N_FRAMES, device="cuda", collect_parity=N_PARITY)
        results["fps_b"] = fps_b
        results["b_device"] = dev
        b_worked = True
        print("[Q2b] processed {} frames  device={}  fps = {:.2f}".format(n_b, dev, fps_b),
flush=True)
    except Exception:
        print("[Q2b] torchcodec device=cuda FAILED:", flush=True)
        traceback.print_exc()
        print("[Q2b] retrying torchcodec device=cpu (decode on CPU, resize/normalize on
GPU)...", flush=True)
        try:
            fps_b, parity_b, n_b, dev = run_torchcodec_frontend(
                model, N_FRAMES, device="cpu", collect_parity=N_PARITY)
            results["fps_b"] = fps_b
            results["b_device"] = dev
            b_worked = True
            print("[Q2b] (cpu-decode fallback) processed {} frames  fps = {:.2f}".format(n_b,
fps_b), flush=True)
        except Exception:
            print("[Q2b] torchcodec cpu fallback ALSO FAILED:", flush=True)
            traceback.print_exc()

    # ---- Q2c: PyNvVideoCodec ------------------------------------------
    print("\n" + "-" * 78, flush=True)
    print("[Q2c] GPU-resident PyNvVideoCodec (DLPack -> cuda) -> GPU resize/normalize -> HRNet",
flush=True)
    print("-" * 78, flush=True)
    parity_c = {}
    c_worked = False
    try:
        fps_c, parity_c, n_c, api = run_pynvvideocodec_frontend(model, N_FRAMES,
collect_parity=N_PARITY)
        results["fps_c"] = fps_c
        results["c_api"] = api
        c_worked = True
        print("[Q2c] processed {} frames  api={}  fps = {:.2f}".format(n_c, api, fps_c),
flush=True)
    except Exception:
        print("[Q2c] PyNvVideoCodec FAILED:", flush=True)
        traceback.print_exc()

    # ---- Q3: parity CPU vs whichever GPU variant worked -----------------
    print("\n" + "-" * 78, flush=True)
    print("[Q3] Quality parity: CPU vs GPU (heatmap deltas + argmax pixel distance)",
flush=True)
    print("-" * 78, flush=True)
    gpu_parity, gpu_name = None, None
    if b_worked and parity_b:
        gpu_parity, gpu_name = parity_b, "torchcodec(b)"
    elif c_worked and parity_c:
        gpu_parity, gpu_name = parity_c, "PyNvVideoCodec(c)"

    if parity_cpu and gpu_parity is not None:
        q3 = compare_parity(parity_cpu, gpu_parity)
        results["q3"] = q3
        results["q3_gpu_variant"] = gpu_name
        if q3 is None:
            print("[Q3] No overlapping frames to compare.", flush=True)
```

```
        else:
            print("[Q3] CPU vs {} over {} current-frames:".format(gpu_name, q3["n"]),
flush=True)
            print("      heatmap mean|delta| = {:.6e}".format(q3["hm_mean_abs"]), flush=True)
            print("      heatmap  max|delta| = {:.6e}".format(q3["hm_max_abs"]), flush=True)
            print("      argmax median px dist = {:.3f}".format(q3["argmax_median_px"]),
flush=True)
            print("      argmax    max px dist = {:.3f}".format(q3["argmax_max_px"]),
flush=True)
    else:
        print("[Q3] Skipped -- need CPU parity data and at least one working GPU variant.",
flush=True)

    _print_summary(results)


def _print_summary(r):
    print("\n" + "=" * 78, flush=True)
    print("SUMMARY", flush=True)
    print("=" * 78, flush=True)
    print("Q1 (build + load weights): {}".format("PASS" if r["q1_pass"] else "FAIL"),
flush=True)
    print("Q2 fps table:", flush=True)

    def fmt(v):
        return "{:.2f}".format(v) if isinstance(v, float) else "n/a (failed)"

    print("   (a) CPU cv2            : {}".format(fmt(r["fps_a"])), flush=True)
    print("   (b) torchcodec [{}]{}: {}".format(
        r["b_device"] or "?", " " * max(0, 6 - len(str(r["b_device"] or "?"))),
fmt(r["fps_b"])), flush=True)
    print("   (c) PyNvVideoCodec     : {}".format(fmt(r["fps_c"])), flush=True)
    if r["q3"]:
        print("Q3 (CPU vs {}):".format(r["q3_gpu_variant"]), flush=True)
        print("   heatmap mean|delta|   : {:.6e}".format(r["q3"]["hm_mean_abs"]), flush=True)
        print("   argmax median px dist : {:.3f}".format(r["q3"]["argmax_median_px"]),
flush=True)
    else:
        print("Q3: n/a", flush=True)
    print("=" * 78, flush=True)


if __name__ == "__main__":
    main()
```

Notes / assumptions / what to check

Normalization (confirmed from repo, load-bearing):
`src/dataloaders/img_transforms.py::build_img_transforms` composes `T.ToTensor()` then `T.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])`. `ToTensor` does the `/255` scale, HWC?CHW, and keeps RGB order. Input frames are read RGB by `utils/utils.py::read_image` (`Image.open(...).convert('RGB')`). So the pipeline is: **RGB ? /255 ? subtract ImageNet mean ? divide ImageNet std**. All three variants implement exactly this.

Which heatmap channel ? position (confirmed): `src/datasets/badminton.py` (lines 189?195) builds a window at index `i` from frames `[i, i+1, i+2]` with targets `ball_xyvs[i : i+3]`, so **output channel `j` corresponds to input frame `i+j`**. For a stride?1 3?frame window, the middle frame is **channel index 1** ? matching the prompt's assumption. `CURRENT_CHANNEL = 1` is a single constant at the top if you want to change it. Note: the official eval does NOT pick a single channel ? it maps every (batch, element) pair back to its own frame path and lets the `OnlineTracker` pick the best detection per frame. This spike intentionally simplifies to "middle channel per stride?1 window," which is the correct per?frame semantic but not byte?identical to the tracker's output.

Sigmoid: `src/detectors/postprocessor.py::run` calls `preds.sigmoid_()` before decoding. HRNet's `final_layers` output is raw logits, so the script applies `torch.sigmoid` before argmax/heatmap comparison. Q3 deltas are therefore on sigmoid?probability heatmaps in [0,1].

Resize is really an affine warp ? but equivalent for 16:9. WASB does not stretch?resize; `dataset_loader.py::get_transform` uses `get_affine_transform` with center `(w/2, h/2)` and scale `max(h,w)` then `cv2.warpAffine(..., INTER_LINEAR)`. This is an aspect?preserving centered letterbox into 512×288. For 1920×1080 (16:9) the box maps exactly to 512×288 (1920/1080 = 512/288 = 1.777?), so it reduces to a plain bilinear resize with no padding. Since g1.mp4 is 1080p 16:9, the script's plain resize (cv2 `INTER_LINEAR` on CPU, `F.interpolate(..., bilinear, align_corners=False)` on GPU) is numerically the intended preprocessing. **If g1.mp4 is ever not exactly 16:9**, replace the resize with the affine warp to keep parity with WASB, and map argmax back through the inverse transform.

Model build: instantiates WASB's own `models.hrnet.HRNet` (not reimplemented) with params copied verbatim from `src/configs/model/wasb.yaml`, passed as an `OmegaConf.create(...)` DictConfig ? the ctor uses both `cfg['...']` and `cfg.MODEL.EXTRA...` attribute access, and OmegaConf supports both, avoiding Hydra composition. `DECONV.NUM_DECONVS=0`, so the two deconv config accesses that would otherwise touch `cfg.MODEL.EXTRA.DECONV.KERNEL_SIZE` are not exercised (the loop body never runs). Weights loaded via `torch.load(..., weights_only=False)` then `["model_state_dict"]` ? `load_state_dict(strict=True)`.

Things to check when it runs on the VM:
- **Q1 is the key result.** If HRNet build or `strict=True` load raises, the full traceback prints and the script exits before Q2/Q3. A key?mismatch here would mean the hard?wired `wasb.yaml` params don't match the checkpoint (they should ? 428 tensors expected; the script prints the loaded count).
- **`sys.path`:** the repo's modules import as top?level (`import models`, `from utils import ...`), so `SRC_DIR` (`~/t2/WASB-SBDT/src`) is prepended to `sys.path`. Run from anywhere; no `cd` needed.
- **torchcodec cuda:** if `VideoDecoder(device="cuda")` fails (NVDEC/FFmpeg build mismatch), the script auto?retries `device="cpu"` (decode on CPU, resize+normalize still on GPU) and labels which path ran in the summary.
- **PyNvVideoCodec API drift:** the `SimpleDecoder` / `OutputColorType.RGB` / DLPack surface varies by version. The variant is defensive (RGB request with fallback, index?or?iterate, HWC?or?CHW handling) but is the most likely to need a small tweak ? if it fails, the traceback shows the exact API gap. If surfaces come back NV12 instead of RGB, a color?convert step must be added before `gpu_normalize_uint8_frames`.
- **Determinism / cudnn:** heatmap Q3 deltas won't be exactly 0 even for identical inputs because cv2 `INTER_LINEAR` and torch `bilinear` use different sampling/rounding; expect small but non?zero `mean|?|` and typically 0?1 px argmax distance if decode+resize are faithful. A large argmax distance signals a real decode/color/resize discrepancy (e.g., BGR/RGB swap or NV12 mishandling in variant c).
- `BASELINE` (`g1_traj_baseline.csv`) is wired as a path constant but not consumed ? Q3 compares CPU vs GPU internally. Extend `compare_parity` against the baseline CSV if you want absolute?vs?baseline validation too.